IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 20 August 2025, accepted 5 September 2025, date of publication 16 September 2025,
date of current version 29 September 2025.

*Digital Object Identifier 10.1109/ACCESS.2025.3610616*

SURVEY

# Binary Code Analysis for Cybersecurity: A Systematic Review of Forensic Techniques in Vulnerability Detection and Anti-Evasion Strategies

**HASEEB JAVED[1], (Member, IEEE), FARMAN ALI[2], BABAR SHAH[3], AND DAEHAN KWAK[4], (Senior Member, IEEE)**

[1]Department of Computer Science and Engineering, Sungkyunkwan University, Seoul 03063, South Korea
[2]Department of Applied Artificial Intelligence, Sungkyunkwan University, Seoul 03063, South Korea
[3]College of Technological Innovation, Zayed University, Dubai 19282, United Arab Emirates
[4]Department of Computer Science and Technology, Kean University, Union, NJ 07083, USA

Corresponding authors: Farman Ali (farman0977@skku.edu) and Daehan Kwak (dkwak@kean.edu)

This work was supported by the Regional Innovation System and Education (RISE) through the Seoul RISE Center, funded by the Ministry of Education (MOE), and the Seoul Metropolitan Government (2025-RISE-01-018-01).

**ABSTRACT** Binary code analysis is essential in modern cybersecurity, examining compiled program outputs to identify vulnerabilities, detect malware, and ensure software security compliance. However, the field faces significant challenges due to the fragmented nature of existing research and the lack of unified analytical frameworks, which hinder comprehensive understanding and practical application. To address these gaps, we conducted a systematic review of binary code analysis techniques across six key areas, analyzing 236 research papers published between 2007 and 2025. This review provides: 1) a comprehensive overview of methods for binary code similarity; 2) a detailed examination of binary code fingerprinting techniques across various scenarios, from malware detection to digital forensics; 3) a systematic review of vulnerability analysis methods, including control flow graphs, taint analysis, and symbolic execution; 4) an assessment of clone detection strategies, such as text-based, token-based, structural, and behavioral approaches; 5) an in-depth study of authorship attribution techniques, with emphasis on malware attribution methods used in real-world cybersecurity cases; and 6) a thorough review of evasion and anti-analysis strategies, along with their countermeasures. In addition to highlighting the strengths and applications of these approaches, the study identifies limitations in current methods, including challenges in malware analysis, vulnerability analysis, and authorship attribution. Finally, it outlines future research directions, including the development of more robust analytical tools, enhancements to attribution models, and the creation of scalable solutions. Overall, this survey provides a foundation for advancing binary code analysis and fostering innovation to enhance software security and resilience by leveraging insights from previous research.

**INDEX TERMS** Code analysis, software security, vulnerability detection, code similarity analysis, cybersecurity.

## I. INTRODUCTION

Binary code analysis is crucial for security testing, as it involves inspecting compiled program outputs to identify potential vulnerabilities. This process, often called"program slicing," consists in examining and assessing software to detect bugs and errors [1]. The goal of binary code analysis is to find and fix errors in the source code before a program goes live. It can assist in understanding how effectively the software handles issues such as deadlocks, memory leaks,

The associate editor coordinating the review of this manuscript and approving it for publication was Porfirio Tramontana.

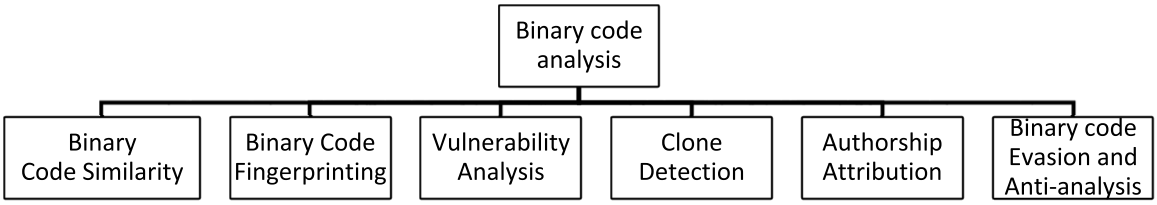



**FIGURE 1.** Key focus areas in binary code analysis.

and infinite loops. An analysis is done to ensure that the code is correct, secure, and meets its specifications. To cite security experts here: "Binary code analysis, often known as binary analysis, is a vulnerability testing technique used for threat assessment performed on binary code. Such analysis examines the raw binary files that make up a whole application, and it is very useful whenever the source code is not available" [2].

Most security applications, such as performance analysis, debugging, reverse engineering, software vulnerability, security analysis, and digital forensics, use binary code analysis [3], [4].

When source code is unavailable and only the program binaries are accessible, especially for many commercial off-the-shelf (COTS) programs, it becomes an essential tool for these applications.

Since research has shown that the semantic analysis of binary code can differ from that of source code, conducting such studies is required [5], [6]. Traditional binary analysis depends mainly on reverse engineering, a time-consuming, error-prone, and expensive manual technique. It's made more difficult by the fact that a significant quantity of semantic information, such as the function concept, comments, and buffer properties, is wasted during compiling [7], [8], [9].

Additionally, binary code analysis has undergone significant evolution over the years, originally stemming from debugging and reverse engineering practices. It centers on analyzing compiled binary code, taking advantage of the fact that most modern computer programs are compiled from high-level languages into intermediate representations with straightforward and consistent structures. This method can simplify and enhance the precision of analysis. It involves inspecting the binary code of programs to find properties related to the source program, such as memory use, timing features, or security aspects [10], [11]. Hence, it is a process of analyzing binary code (also called machine code, instruction code, or executable code) to determine what it does and how it works [12], [13]. It is also helpful in discovering security vulnerabilities in software because hackers can exploit bugs in the binary code [14], [15], [16]. The review paper was conducted through a systematic and comprehensive literature search, covering publications from 2007 to 2025, which included 236 relevant research papers.

The systematic literature review methodology was adopted, and rigorous selection criteria were applied for this review. We collected and analyzed 236 research papers (2007–2025) using a systematic literature review methodology, including Google Scholar, IEEE Xplore, ScienceDirect, ACM Digital Library, SpringerLink, and Wiley Online Library. These databases were selected based on their extensive coverage in the field of binary code analysis, covering computer science, software engineering, and cybersecurity. The search process involved several keywords and phrases, such as"binary code similarity,""software vulnerability detection,""vulnerability detection,""binary code clone detection," and"authorship attribution in binary code," to identify the most relevant literature. The review offers a critical analysis of binary code analysis techniques and their applications across six key areas as shown in Figure 1: binary code fingerprinting, vulnerability detection, similarity analysis, authorship attribution, clone identification, and methods for countering evasion and anti-analysis.

### A. CONTRIBUTIONS

This study presents a comprehensive and pioneering assessment of current binary code analysis practices, with a focus on their applications and recent developments in the field. The main contributions of this review are summarized below and listed in Table 1:

1. This study reviews and categorizes existing methods for binary code similarity, aiding researchers in understanding current techniques for evaluating similarity in compiled programs.
2. Building on similarity analysis, this paper reviews binary code fingerprinting methods, categorizing applications from malware detection to digital forensics, emphasizing their importance and potential research areas.
3. Complementing these identification methods, this work reviews and categorizes the key features commonly used in vulnerability detection approaches through comprehensive literature analysis, providing insights into their effectiveness and application contexts in binary code security analysis.
4. Besides vulnerability detection, the study reviews existing clone detection methods in binary code analysis, examining their core principles, implementation strategies, and real-world uses across various research settings.

**TABLE 1.** Comparative analysis of related review studies in binary code analysis.

| Study | Published Year | Number of reviewed papers | Literature coverage range | Binary Code Similarity | Binary Code Finger-printing | Vulnerability Analysis | Clone Detection | Authorship Attribution | Binary code Evasion and Anti-analysis | Study theme |
|---|---|---|---|---|---|---|---|---|---|---|
| **Ours** | 2025 | 239 | 2007-2025 | Yes | Yes | Yes | Yes | Yes | Yes | Investigation of emerging binary code analysis |
| [17] | 2024 | 37 | 2009-2024 | Yes | No | Yes | No | No | Yes | Binary code analysis datasets |
| [18] | 2024 | 26 | 2015-2024 | No | No | Yes | No | No | Yes | Malware analysis and detection |
| [19] | 2024 | 149 | 2008-2024 | No | No | Yes | No | No | No | AI-based ransomware detection |
| [20] | 2023 | 65 | 1987-2023 | Yes | No | No | No | No | No | Binary Code Similarity Detection |
| [21] | 2023 | 32 | 2013-2023 | No | No | Yes | No | No | Yes | Binary Code Vulnerability Detection |
| [22] | 2023 | 150 | 2015-2023 | No | No | Yes | No | No | No | Ransomware and Vulnerability Analysis |
| [23] | 2022 | 192 | 2007-2022 | No | No | Yes | No | No | Yes | Evasion and Counter-Evasion in Malware Analysis |
| [24] | 2022 | 136 | 1996-2023 | Yes | No | No | Yes | No | No | Clone detection and source code similarity |
| [25] | 2022 | 31 | 2001-2022 | Yes | No | No | No | No | No | Source Code Embeddings Study |
| [26] | 2022 | 52 | 2011-2022 | No | No | Yes | No | No | No | Mining Vulnerabilities in Binary Code |
| [27] | 2021 | 116 | 2013-2021 | No | No | Yes | No | No | No | IoT Firmware Vulnerability Detection |
| [28] | 2021 | 209 | 2007-2021 | No | Yes | Yes | Yes | Yes | No | Binary Code Fingerprinting Approaches |
| [29] | 2021 | 99 | 1986-2021 | No | No | No | Yes | No | No | Software Clone Detection |
| [30] | 2021 | 56 | 1985-2021 | No | No | No | No | No | Yes | Evading Binary Code |
| [31] | 2020 | 45 | 2014-2020 | No | No | Yes | No | No | No | Techniques for Detecting Mobile Malware |
| [32] | 2019 | 241 | 2004-2019 | No | No | Yes | No | No | No | Outlier detection techniques |
| [33] | 2019 | 93 | 1997-2019 | No | No | No | No | Yes | No | Code Authorship Attribution |
| [34] | 2019 | 104 | 2007-2019 | No | No | Yes | No | No | No | Dynamic malware analysis |
| [35] | 2018 | 52 | 2014-2018 | No | No | Yes | No | No | No | Malware detection approaches |
| [36] | 2018 | 43 | 2013-2018 | No | No | Yes | No | No | No | Malware Analysis and Detection |
| [37] | 2018 | 26 | 1978-2017 | No | No | No | No | Yes | No | Programs Authorship Attribution |
| [38] | 2018 | 125 | 1991-2018 | No | No | Yes | No | No | Yes | Evasion Techniques in Dynamic Malware Analysis |
| [39] | 2017 | 65 | 2007-2017 | Yes | No | No | No | No | No | Utilizing Fuzzing to Analyze Binary Code Similarity |
| [40] | 2016 | 115 | 1992-2016 | No | No | No | Yes | No | No | Code clone detection |
| [41] | 2013 | 89 | 2009-2013 | No | No | Yes | Yes | No | No | Binary-code obfuscations |
| [42] | 2013 | 236 | 2001-2013 | No | No | No | Yes | No | No | A survey of software clone detection |
| [43] | 2009 | 110 | 1964-2009 | No | No | No | No | Yes | No | Authorship attribution methods |
| [44] | 2007 | 225 | 1991-2007 | No | No | Yes | Yes | No | No | Clone detection research |
| [45] | 2007 | 360 | 1991-2007 | No | No | Yes | No | No | No | Outlier detection |
| [46] | 2007 | 119 | 1995-2007 | Yes | No | No | No | No | Yes | Source code analysis |

5. Finally, this paper reviews the authorship attribution techniques and evasion methods in binary code analysis, focusing on their development, effectiveness, and real-world use cases in cybersecurity investigations, which highlight the advanced analytical challenges.

### B. ARTICLE STRUCTURE

The following paragraph outlines the structure of this study. Section II covers binary code analysis, its importance, survey objectives, selection process, and applications like code similarity and fingerprinting. Section III examines vulnerability analysis in binary code, detailing various techniques and methods used to identify and evaluate vulnerabilities in software systems. Section IV highlights the significance of clone detection in code maintenance and software development. Section V discusses authorship attribution in binary code analysis, focusing on methods used to determine the origin or author of a specific piece of code. It also explores techniques used by attackers to obfuscate code and evade detection by security systems and analysts in Section VI. The final section of the paper addresses the limitations of binary code analysis.

## II. BACKGROUND: BINARY CODE ANALYSIS AND DISCOVERY

Binary code analysis serves as a fundamental technique in understanding and defending against advanced cyber threats in computing environments. This section outlines the core principles of binary code analysis and their application in cybersecurity. It explains why analysts need to study binary analysis, which can help identify code patterns and understand malware behavior. Furthermore, we discuss key methodologies and techniques used in analyzing executable files.

### A. IMPORTANCE OF BINARY CODE ANALYSIS

Malware binaries contain valuable information related to their digital architecture, code characteristics, and other semantic details, including timestamps. In a study conducted by FireEye experts, they observed that a particular malware family exhibited similarities in its infrastructure [47]. This discovery significantly enhances the training of forensic experts in code and binary analysis, enabling them to identify specific malware families with greater confidence. It emphasizes the importance of understanding the technical details

of malware binaries, which facilitates improved detection and attribution of malicious activities in cybersecurity [48]. This section lays the foundation for binary code analysis and discusses its key applications in cybersecurity. The following subsections explore the primary application areas where binary code analysis is essential, highlighting its practical importance in various cybersecurity contexts.

#### 1) MALWARE ANALYSIS

When malware infects a computer network or system, it typically releases an executable rather than the source code. In the ongoing battle between malware authors and anti-malware providers, a solid understanding of binary analysis is crucial. Depending on its goals, each side performs various activities on the binary code. The Chernobyl virus, for instance, might be identified by searching for a specific type of hexadecimal sequence [49]:

$$E800\ 0000\ 005B\ 8D4B\ 4251\ 5050$$
$$0F01\ 4C24\ FE5B\ 83C3\ 1CFA\ 8B2B$$

As a result, the malware may attempt to make the binary code more complex to evade anti-virus detection. Even though the signature-based approach is effective at detecting known malware, it is not capable of identifying unknown or zero-day malware variants, including ransomware [50].

#### 2) DIGITAL FORENSICS

According to research conducted by FireEye, malware binaries carry vital information related to their digital architecture, code characteristics, and semantic details, including timestamps. In computational findings, FireEye specialists determined that a unique family of malware exhibited patterns in its infrastructure [51]. These findings are of great importance in offering critical exercises to forensic specialists in code and binary analysis.

Digital forensics is a method used to determine what happened to a computer system by examining data. It involves searching for clues, such as deleted or hidden files. The data needed for digital forensic analysis can come from various sources [52], [53], [54]. One of these sources includes information stored in binary code. Binary code is a sequence of 0s and 1s that represent instructions to a computer [54]. This can be associated with a sequence of systematically organized instructions, although written in 0s and 1s rather than words and images. When executing a program on a computer, it converts the binary code into understandable information, such as the text displayed on this page [55].

However, without the original binary file, retrieving this information can be difficult. Sometimes, the only data available are sequences of ones and zeros. This is where digital forensics comes in. Digital forensics experts examine binary files to determine what type of data may be contained within. They are like detectives investigating crimes that happen online or with computers [56], [57], [58]. In addition to forensic investigations, analyzing binary code also deals with legal and intellectual property issues related to software development and distribution.

#### 3) COPYRIGHT INFRINGEMENT

Copyright infringement in binary code analysis refers to the unauthorized use, distribution, or reproduction of copyrighted binary code without the explicit permission of the copyright holder. Binary code analysis involves the examination and understanding of software programs represented in machine-readable format [59]. Two or even more writers may cooperate in forensics apps to adjust the code styles to fit different styles [60]. This stresses the value of binary analysis in identifying hints that aid in the resolution of such situations [61]. For example, malware samples (such as Zeus) typically contain both copyright-protected and non-copyrightable elements. As a result, creating an online database of potential authors compared to previously gathered malware samples may aid in infringement investigations. On the other side, it has been demonstrated that software piracy will affect a large number of customers [62].

Copyright infringement claims can be brought against entities that reproduce or disseminate copyrighted works without permission [63], [64]. Typically, these claims involve allegations that someone has copied a literary, artistic, or musical work. But what about computer programs? Programs are considered literary works under copyright law, but their composition differs. Computer programming languages are essentially methods for converting concepts into binary code in a format that computers and other devices can understand. The same program written in two different programming languages will produce two sets of binary code that look entirely different [65], [66].

To avoid potential legal issues when using BCA on software you do not own, ensure the program has an open-source license. If no permission exists, then assume it is proprietary and contact the copyright holder before attempting any BCA activities. If an open-source license is available but no explicit permission is granted, do not use BCA on their code unless they explicitly allow it. If both conditions are met, make sure all results from any testing done by BCA are published so others can benefit from them too [67]. Alongside software analysis, examining binary code also involves hardware-embedded systems, tackling security issues in IoT devices and embedded platforms.

#### 4) FIRMWARE ANALYSIS

Firmware analysis is a crucial aspect of binary code examination, involving the study and understanding of firmware embedded in electronic devices, such as microcontrollers, IoT devices, routers, and other hardware components. Firmware acts as the low-level software that manages the functions and operations of these devices [68]. It is also defined as "a type of software that provides control, monitoring, and data manipulation of engineered products and systems." Firmware is software or a set of instructions installed on hardware [69].

It contains instructions about whether the device interacts with the rest of the computer hardware [70]. In other words, firmware acts as the connection between software and hardware [71]. Firmware is usually stored in a device's read-only memory (ROM). This prevents accidental modification or deletion, which could cause the device to stop working [72]. Having established these foundational applications, the next section will explore the specific methodologies and techniques employed in binary code similarity analysis.

### B. BINARY CODE ANALYSIS VS BINARY CODE SIMILARITIES

Binary code analysis involves a thorough examination of machine-level instructions and data within a software program's binary representation, aiming to understand its behavior and identify vulnerabilities [72]. On the other hand, binary code similarities involve comparing multiple binary files to identify patterns and relationships, which is useful for tasks like malware classification and code plagiarism detection. Both methods are essential in cybersecurity and software analysis, offering valuable insights into software functionality and security issues [73]. The number of lines of code used in each program may also be important in determining whether or not it is possible to determine whether a program was written using one programming language or another [74], [75]. Both binary code analysis and binary code similarity are forms of reverse engineering. However, while binary code analysis helps in understanding how something works, binary code similarity takes that one step further [76].

Binary code similarity methods involve the comparison of segments of binary code, as described by [77]. Three key features continue to characterize these methods: (1) the type of comparison, such as determining if segments are the same, comparable, or equal; (2) the precision of the binary code bits being analyzed, such as basic blocks, instructions, or functions; and (3) the combination of multiple components being evaluated, which can be one-to-many, one-to-one, or many-to-many, as discussed in [78]. To simplify the discussion, we first focus on the comparison type and granularity when examining two inputs, then expand the discussion to include multiple inputs. This method enables a deeper examination of the three core traits that underpin binary code similarity methods.

#### 1) COMPARISON TYPE

In binary code analysis, various forms of comparison can be utilized to identify similarities between different segments of binary code, such as representing the code as hexadecimal strings for raw bytes, dismantled instructions, or control-flow graphs [79]. The comparison process typically involves making a simple Boolean decision to determine if the binary digits are similar or not. This can be achieved by computing a hash code (e.g., SHA256) for each component's content and comparing the resulting hashes to ascertain their similarity. The components are equivalent if the hashing is the same. Nevertheless, in many circumstances, even a simple technique fails to find a resemblance [80]. If you have ever compiled a program, you know it's a process that can be affected by many different factors. When compiling a program, the source code, which is a text file written in a programming language and saved with an extension such as '.c' or '.java', is converted into binary code, a series of 1s and 0s that the computer can understand. Even if the same source code is compiled twice using all the same settings, two different binary files with different hashes will still be produced, because metadata included in the executable's header may differ between compilations [81], [82].

To grasp semantic equivalence in binary code, it is essential to recognize that if two binary bits have identical semantics, meaning they execute the same functions, they are deemed comparable. The structural arrangement of the binary code is irrelevant in determining equivalence. Hence, two identical binary bits would exhibit similar semantics, while different segments of binary code have the same meaning [78].

For example, *mov %eax*, $0 *and xor %eax*, *%eax* have both changed the value of register EAX to zero; they are semantically equivalent x86 instructions. Likewise, the exact source code built for two distinct target architectures should generate comparable executables, even if the architectures' instructions are entirely different [83]. That semantic similarity, on the other hand, extends beyond the binary code produced from the very similar source code. Two implementations of a similar encryption algorithm, for instance, must be semantically comparable. Demonstrating that two random programs are functionally similar is an intractable challenge that can be reduced to a halting problem. In reality, evaluating binary code equivalency is a time-consuming procedure that can only be done for tiny amounts of binary data [84], [85].

#### 2) COMPARISON GRANULARITY

In the context of binary code similarity methods, the comparison granularity can vary, encompassing instructions, basic blocks, or even entire programs. Certain approaches adopt a two-step process, initially conducting a fine-grained comparison and then aggregating the results to facilitate a broader assessment. For instance, a technique might measure the percentage of identical procedures between two programs to ascertain their similarity. It is essential to distinguish between the granularity of the inputs, which refers to the specific chunks of binary code analyzed by the method, and the granularities of the measurements used within the technique itself. This consideration enables a more nuanced and accurate assessment of binary code similarities. Applying precise contrast at a finer level of detail could limit the types of comparisons possible at a coarser level [82].

It demonstrates that determining if two bits of binary code are similar at a finer granularity (for example, their basic blocks) may be used to determine if the finer-grained parts that include them (for example, their functions) are similar, identical, or comparable. Comparison at a smaller granularity,

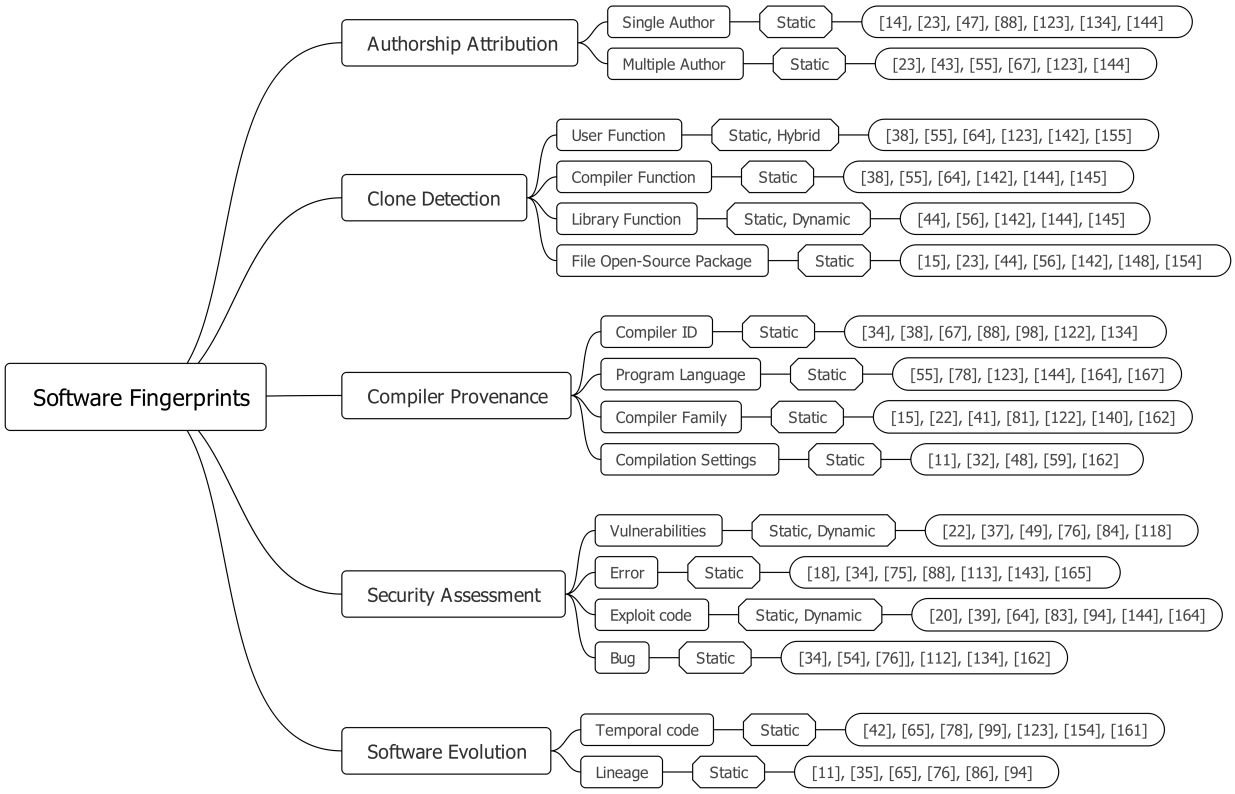



**FIGURE 2.** Applications of software fingerprinting in binary code analysis.

on the other hand, cannot be utilized to conclude that the coarser granularity code is equal or the same [86]. When comparing two functions, for instance, that all their basic blocks are comparable does not mean they are equal or equivalent. Comparison, on either side, is the most generic sort of competition, and it may be inferred to use some finer-granularity comparative type [87].

#### 3) NUMBER OF INPUTS

Identical binary codes with two or more bits can be compared using different methods. Those who contrast more than two components can be divided into two groups: those who evaluate one piece to the others and those who compare each element to specific other components. As a result, we distinguish three types based on the number of inputs and their comparison: one-on-many (OM), one-on-one (OO), and many-on-many (MM). The program determines the sources of the incoming bits. They might come from the same program version (e.g., two sections of the identical executable), two separate programs, or two distinct versions of the same software [88], [89].

A querying bit of binary code is compared to multiple target portions of binary digits in one-to-many techniques. Most OM techniques employ binary code search, which means they examine whether the query component matches all the other target parts and then return the top-most comparable binary code objective pieces. The target parts may originate from various versions of the same program (excluding the query piece), separate programs written for the same platform, or distinct programs coded for different architectures [67]. Many-to-many techniques do not discriminate among source and destination components, unlike OO or OM systems. All incoming elements are treated equally and thus are compared to one another. These methods often execute binary code clustering, i.e., typically produce groupings of binary code that are comparable. Figure 2 illustrates the concept of fingerprinting binary code applications, showcasing the diverse applications and implications of this technique. Through fingerprinting, binary code can be uniquely characterized, allowing for authorship attribution, which involves identifying the origin or creator of specific code segments.

### C. BINARY CODE FINGERPRINTING

Fingerprinting is a technique that allows websites to recognize returning visitors. Binary code fingerprinting works by having JavaScript code on the web page ask a series of questions about the user's computer and browser settings, such as which plugins are installed or the width of the browser window. By using binary code fingerprinting and leveraging known malware datasets such as VirusTotal, Malicia, and BIG2015, companies can identify known malware at the moment of infection and stop it before it spreads any further [90], [91], [92], [93]. The components involved in binary code fingerprinting are summarized in the following table,

**TABLE 2.** Comprehensive analysis of binary code fingerprinting techniques and their applications.

| Components | Usage Cases | | | | | Multiple Challenges | | | | References |
|---|---|---|---|---|---|---|---|---|---|---|
| | Digital Forensics | Website Fingerprinting | Malware Analysis | Firmware Analysis | Copyright Infringement | Compiler Effects | Function Boundary | Semantic Deficiency | Disassembling | |
| Importance of Fingerprinting | | | ✓ | | | | ✓ | | | [96] |
| | ✓ | | | | | ✓ | ✓ | ✓ | | [97] |
| | ✓ | | ✓ | | | ✓ | ✓ | | | [98] |
| | | | | | ✓ | | | ✓ | | [99] |
| | | | | ✓ | | ✓ | ✓ | ✓ | ✓ | [100] |
| | | | ✓ | | | ✓ | | | | [101] |
| | ✓ | | | | | | ✓ | | | [102] |
| | | ✓ | ✓ | ✓ | | | | | | [103] |
| | ✓ | | | | | | | | | [104] |
| **Applications of Binary Code Fingerprinting** | | | | | | | | | | |
| Problems of Binary Code Fingerprinting | Clone Detection | Code Obfuscation | Code Compression | Software Evaluation | Android Malware | VA and LFF | Fingerprinting Analysis | | | References |
| | | | | | | | Static | Dynamic | Hybrid | |
| | ✓ | | | | | | ✓ | | | [105] |
| | | ✓ | | | | ✓ | ✓ | | | [106] |
| | | | | | | ✓ | | ✓ | | [107] |
| | ✓ | | | | | ✓ | | | ✓ | [108] |
| | | | ✓ | | | | ✓ | | | [109] |
| | | | | | | | ✓ | | | [110] |
| | | ✓ | | ✓ | | ✓ | ✓ | | | [111] |
| | | ✓ | | | ✓ | | ✓ | ✓ | | [112] |
| **Reverse Engineering Tools and Frameworks** | | | | | | | | | | |
| Extraction of Fingerprinting | Pelican | F-ACCUMUL | EnviRule | BINO | Angr | ROPgadget | Triton | Fingerprinting Process | | References |
| | | | | | | | | FG | FM | |
| | | | | ✓ | | | | | | [113] |
| | ✓ | | | | | ✓ | | | | [114] |
| | | | | | ✓ | | | | | [115] |
| | | ✓ | | | | | ✓ | ✓ | | [116] |
| | | | | | | | | | ✓ | [117] |
| | | | ✓ | | | | | | | [118] |

along with their various applications and the associated challenges. These elements are essential for multiple applications, including software analysis, intellectual property protection, digital forensics, and cybersecurity. Understanding the uses and challenges of these components is vital for solving complex real-world problems and advancing research in related fields.

Fingerprinting enables clone detection, identifying duplicated code fragments within software. Binary code fingerprinting helps detect known malware, preventing damage. It is vital for cybersecurity, stopping attacks early and preventing threats. All organizations will likely adopt this tool to strengthen security and defend assets from sophisticated cyber threats. This widespread adoption is likely to result in stronger defenses across various industries, thereby building greater trust and stability in digital operations. As technology advances, integrating such tools will be crucial for staying competitive and protecting sensitive information [94], [95]. To do this, a review of some research has been done, as shown in Table 2:

To help protect users, companies employ fast and effective binary code fingerprinting to detect malware infections early and prevent their spread. By using binary code fingerprints, companies can identify malware already in the system, stopping it from spreading further. Binary code fingerprinting is crucial for security, helping companies advance in cybersecurity [119], [120]. However, binary code fingerprinting can be explained mathematically using hashing, feature extraction, and similarity measures. Here are some of the key mathematical concepts for feature extraction, hashing signatures, and similarity measures. Given a binary executable, it extracts features such as opcode sequences, API calls, and control flow graphs to represent as vectors in a high-dimensional space. However, A hash function $H:B\rightarrow Z^m$ maps the binary to a fixed-length signature, and some common methods can be represented as: n-grams, SHA-256, and locality-sensitive hashing (LSH). For Similarity Measures, fingerprints of binaries can be compared using similarity metrics like The Hamming distance for bitwise fingerprints is defined

**TABLE 3.** Comparative vulnerability analysis of anomaly detection.

| References | Type | Approaches | Inside / Cross-projects | Security focused |
|---|---|---|---|---|
| [151] | API usage patterns | Association rule-based mining | Inside | No |
| [152] | API usage patterns | Template rule-based extraction | Inside | Yes |
| [153] | API usage patterns | Frequent close-based itemset mining | Inside | No |
| [154] | API usage patterns | Frequent close-based itemset mining | Inside | No |
| [155] | API usage patterns | Frequent partial-order based itemset mining | Cross | No |
| [156] | Missing check | Maximum frequent sub-graph-based mining | Inside | No |
| [157] | API usage patterns missing check | Imbalance frequency-based itemset mining | Cross | No |
| [158] | API usage patterns | Frequent close-based itemset mining | Cross | No |
| [159] | Missing check | Bag-of-words + k-Nearest neighbor | Inside | Yes |

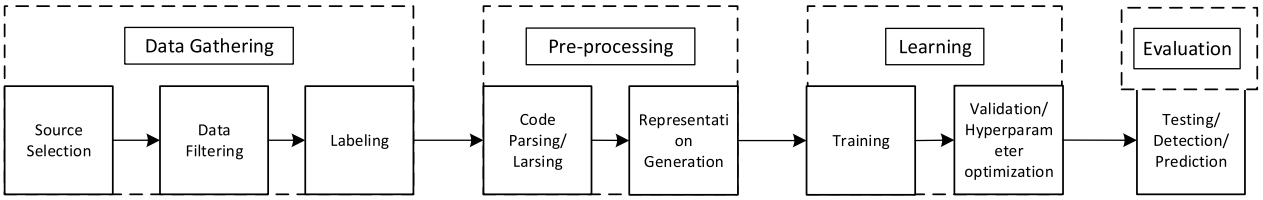



**FIGURE 3.** Comparative vulnerability analysis of anomaly detection.

in equation 1:

$$d_H(A, B) = \sum_{i=1}^{m} (a_i \oplus b_i), \tag{1}$$

where A and B are binary fingerprints, $a_i$ and $b_i$ are the i-th bits of fingerprints A and B, respectively, and m is the fingerprint length. Here, the Hamming distance counts how many bits differ between two binary fingerprints by performing the XOR operation ($\oplus$), operations on corresponding bits and counting the resulting ones. The XOR ($\oplus$) returns 1 if the bits are different and 0 if they are the same, so the total sum reflects the number of mismatched positions. To implement this, compare each bit position iteratively and tally the mismatches using bitwise XOR on the binary fingerprints. Here are some of the applications for fingerprinting binary code. As a result, binary code fingerprints have emerged as a critical step in binary code analysis. The process of extracting fingerprints from binary digits has been thoroughly investigated to gain a comprehensive understanding of its operation and design, which can be utilized to store information in the dynamic characteristics of a computer program, rather than in the program's static file data. Clone detection [121], authorship attribution [122], library function identification [123], vulnerability [124] and bug identification [125], compiler provenance retrieval [126], identification of binary similarities [127], and the allocation of such fingerprint to every function for resulting use by a binary web search are all examples of applications where this method is used. The ability to automate some of the work involved in finding vulnerabilities, undertaking type inference research, and conducting taint monitoring is another advantage of binary code fingerprinting analysis [128].

## III. VULNERABILITY ANALYSIS OF BINARY CODE

One of the most crucial steps in identifying potential vulnerabilities in code is conducting a vulnerability binary scan. The initial step should be to determine which assets are most critical to the company and ensure they are adequately protected. These scans are commonly integrated into development processes to verify that the products being developed meet security requirements and are safe for deployment. Protecting sensitive client data is a fundamental responsibility, emphasizing the need to establish strong scanning procedures to maintain the security and integrity of software systems [129], [130]. Binary code is the language of computers. It is composed of a series of zeros and ones that are easily interpreted by machines but not as easily by humans. To understand binary code, it needs to be converted into a more human-readable language [131], [132], [133]. This conversion is not always an easy job–and it can lead to errors in the code during the conversion process. These errors often result in what is called ''vulnerability,'' which means that hackers will more easily be able to access the computer system if there is a flaw in its conversion from binary code [134], [135], [136]. The best way to prevent this from happening is through manual testing, which involves hiring skilled individuals who can convert binary code into a human-readable format and then review the translated version before making any changes. This guarantees that no errors have occurred during translation; however, it can be costly because human testers need compensation for their time [137].

Binary Code Vulnerability Analysis can be explained using control flow analysis, taint analysis, and symbolic execution. A control flow graph (CFG) depicts a program as a directed graph, with nodes representing basic blocks of sequential instructions and edges illustrating control flow paths. Vulnerabilities, such as buffer overflows, are identified by examining paths within this graph. In taint analysis, define T(x) as a taint function where x is an input variable. When tainted data reaches a sensitive operation, such as system calls, it can trigger vulnerability detection, allowing for improved access controls or automated vulnerability scanning. In symbolic execution, each instruction is converted into a mathematical expression along different program paths, and then it determines whether an attacker can manipulate the program into an unintended state. This approach enables the automatic analysis of programming behavior and the identification of potential vulnerabilities.

Vulnerabilities can also occur in function pointers, which are addresses in memory where instructions are located [138]. Function pointer errors include invalid memory access, improper use within loops or recursion, or uninitialized values; they might even lead to security-critical code being overwritten by an attacker who knows how to exploit them [139]. Software companies are constantly under attack, and to try to combat this, they put significant resources into protecting their product through encryption [140]. While this does a good job of keeping out unwanted users and hackers, it also prevents the software companies from seeing what is going on inside their own code [141]. In the realm of web security, two types of vulnerabilities are both common and highly dangerous [142]. These two types of vulnerabilities, known as SQLI and XSS, are extremely serious because they give hackers access to secure databases and allow them to steal private user data. Anyone involved in site development or digital operations must know how to stop these threats. To help, we have prepared a short tutorial on how to defend against these attacks using XSS and SQLI scanners [143], [144]. However, analyzing program binaries is challenging because much semantic content, such as function prototypes, buffer properties, and source control and data flow graphs, is lost during the formulation phase. The binary code consists of modules, basic blocks, function sequences, instructions, and information. Table 3 provides an overview of existing literature on anomaly detection in vulnerability analysis, highlighting security aspects of these studies.

Malicious actors can use the binary code for an application directly, without altering its functionality, to gain information about its internal workings [145]. They can extract sensitive info like passwords or tokens that the binary code doesn't intend to share [146]. They can insert or overwrite code to alter the application's output, allowing them to change how it works or see hidden information [147]. This kind of vulnerability is called a reverse-engineering attack because it involves changing the program's behavior by looking at its inner workings (which is called reverse-engineering) [148].

In [149], the author evaluates machine learning models in coding across various applications and examines the naturalness of code in their survey. As the number of flaws increases, software security may decline; however, specifically identifying vulnerabilities is a security-related fault that an attacker might exploit. Another key approach in malware analysis emphasizes using machine learning techniques, as recent studies highlight. This method improves detection capabilities by using advanced algorithms to identify malicious patterns more accurately [150].

The work also includes object code analysis, with a focus on vulnerabilities created by engineers rather than malicious software infiltrated because of an attacker. But, in [160], it released a paper on deep learning-based open-source vulnerability identification. Our paper covers some of the same core findings as this one; however, it also goes into object source code and the variations in the underlying deep learning architecture, whereas this paper concentrates on deep learning models [161].

Cheng et al. [162] looked at whether intricacy, code churning, and development activity (CCD) measures might be used to forecast vulnerabilities. The authors accomplished this by conducting empirical case analyses on two open initiatives. This research examined a combination of 28 CCD software measures, comprising 14 complexity indicators, 3 code churn measurements, and 11 programmer data points. The authors utilized Welch's t-test to assess the metrics' discriminating power, ensuring that the test hypothesis was accepted for at minimum 24 of 28 measures with both initiatives. For model validation, the authors employed next-release evaluation when multiple releases were accessible and used cross-validation in other cases. However, a limitation of this study is that presenting results for only one classification algorithm might restrict understanding of the advantages and disadvantages of different modeling methods.

In [66], the authors point out three major flaws in earlier vulnerability forecasting models, and as a result, they present a novel technique for predicting susceptible places in software relying on measured and calculated data that address these flaws. The authors developed a semi-automatic analytical methodology for detecting software vulnerabilities and utilized its results as part of vulnerability research, rather than relying on reported vulnerabilities, claiming that this approach provides more comprehensive information regarding software vulnerabilities. However, the semi-automatic methodology may generate false positives or overlook certain vulnerabilities, limiting the accuracy and completeness of its findings [163], [164], [165], [166], [167]. Unlike prior studies that solely examined in-between vulnerability predictions, this one utilized data from five accessible projects to investigate systematic cross-project risk predictions. The trials employed a variety of classification approaches. At file-level granularity, 11-unit complexity measures and 4 connection metrics were assessed [168], [169].

As a result, DL approaches for vulnerability assessment can better manage an often-fuzzy pattern of software bugs, as well as incorporate natural languages, such as comments and identification names, into the study, which is typically excluded in traditional program analysis [170]. Moreover, because of their statistical nature, DL approaches have the potential to cope better with the ill-defined features of software vulnerabilities, as seen by the frequently reported large false-positive results for traditional program analysis [171]. The DL model's typical training pipeline consists of four primary phases: data collection, pre-processing, and assessment (see Figure 3).

Vulnerabilities represent a subset of all potential software issues that can arise within a program's code [172]. The primary objective of deep vulnerability analysis is to detect known vulnerability types in previously unexplored and novel program code, rather than discovering entirely new types. To illustrate this concept and our discussions in the subsequent sections, we present an example of an open-source software susceptible to integer overflow (Code). The code written in C is based on a command prompt argument that allows the user to focus on providing input. The strtoul ( ) function is used to transform this parameter into an unsigned long integer (32-bit). The result is provided to a test ( ) function, which requires unsigned integers (16-bit) as inputs. Inputs more than 16 bits, such as n > 1, cause an integer overflow mostly at three highlighted points in the code. An integer overflow is classified as a vulnerability type CWE-190, and it can cause the stack to be overwritten if the buffer size is set to be smaller than the amount of data transmitted into it (cp. line 6 of Listing 1).

**Algorithm 1 Sample Source Code From Cui et al. Dataset: Into-Bad1.c [145]**

```
# include <stdio.h>
# include <stdlib.h>
void test(unsigned int n){
   int ∗ buff, j;
   buf = malloc(n ∗ size of ∗ buff); /∗ vulnerable∗ /
   if (!buff) return;
   for (j = 0; j < n; j++)
      buf[j] = i; /∗ vulnerable∗ /
   while (j–> 0)
   printf(''% x'', buff[j]); /∗ vulnerable∗ /
   printf(''\ n'');
   free(buff);
}
int main(int argc, char ∗∗ argv){
   int n;
   if (argc! = 2) return j;
   n = strtoul(argv[j], 0, 10);
   test(n);
   return 0;
}
```

### A. DATA COLLECTION PHASE

The accuracy of predictions made by a Deep model heavily relies on the availability of relevant and well-trained data. For supervised training approaches, large volumes of high-quality labeled training samples are necessary. Fortunately, the increasing popularity and widespread use of collaborative software design have led to an abundance of data available for Deep Learning (DL) on program code, particularly from software forges. However, efficient decision-making and data filtering are essential in distinguishing between numerous unnecessary and experimental initiatives that involve production code and are well-suited for training purposes. The significant imbalance between susceptible and non-vulnerable code in web applications makes it impossible to use genuine program code. As a result, synthetic program code is frequently utilized to counter the scarcity of labeled code samples and their high imbalance.

### B. PRE-PROCESSING PHASE

In code analysis, the pre-processing phase is crucial for preparing code samples for deep learning (DL). Enhance the model's understanding of code structure and semantics. It requires writing and commenting code snippets to emphasize aspects like data types. A special-purpose parser or lexer tokenizes the source code into relevant units such as statements or expressions. The meaningful units are transformed into tokenized formats, such as graphs or assertions, wherein each unit is represented by a single token. After converting to numeric vector form using embedding or encoding methods, they are suitable for deep learning models that process numerical data.

These embeddings allow DL algorithms to capture patterns and make accurate trend forecasts by encoding dependencies and relations between code elements in a continuous vector space. This approach combines representation learning with key feature engineering, ensuring the model receives rich, structured data for effective code analysis and assessment. All in all, pre-processing unification of the learning model and raw code is done by arranging and converting the code into a form that enhances predictive accuracy and learning efficiency.

### C. LEARNING PHASE

In the learning phase, a DL program code vulnerability assessment model is trained using processed data to classify code as either vulnerable or non-vulnerable. This stage involves significant automated variable adjustment, along with precise hyperparameter modification, which impacts model performance. These hyperparameters are changed in a controlled manner within a certain procedure. To ensure a bias-free evaluation, the dataset is typically split into training, validation, and test sets with a common percentage ratio of 80:10:10. The test set evaluates the model's outcome without training, the validation set validates the model, and the training set trains the model. Reliability in assessing

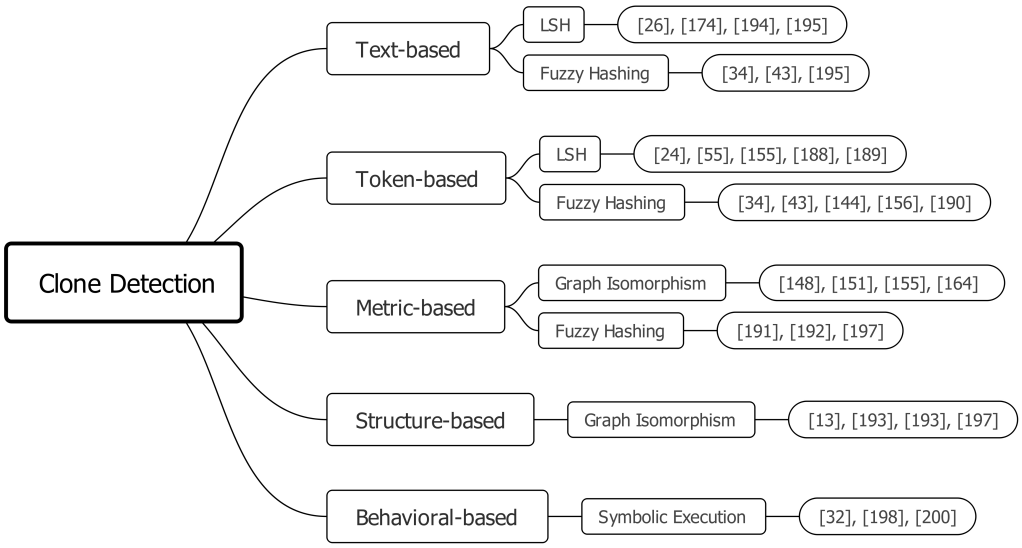



**FIGURE 4.** Multiple approaches to binary code clone detection.

performance, tuning the model, and exposing the varying training is maximized through the efficient implementation of the methodology [173].

### D. EVALUATION PHASE

After training, the model's performance is evaluated using new sets and metrics, or with an autonomous assessment split intended to test generalization and usefulness. To ensure that the model provides relevant predictions for unknown data, the assessment splits the dataset into training and testing segments. Evaluating the model with new datasets is helpful in confirming its performance under varying conditions, biases, or distributions. To verify the model's dependability and suitability for real-world applications, performance benchmarks are also created using criteria provided by industry or research.

## IV. CLONE DETECTION OF BINARY CODE

Binary clone detection refers to the identification of functional equivalence between two binary programs, determining if they perform the same tasks. This process involves collecting usage information and converting it into a feature vector that facilitates automatic comparison of the two programs [174], [175], [176]. Instead of finding all similar code fragments, assembly code clone search in malware analysis concentrates on finding code fragments that are similar to a target [177]. There are four sorts of clones: textual clones (sorts I, II, and III) and semantic clones (Type IV). A code fragment is a group of lines that perform a certain function. The layout, comments, and spacing are the only differences between Type I clones. Variables, constants, and memory references are also different in type II clones. While Type III clones may have additional, changed, or missing statements, they nonetheless exhibit some degree of resemblance. Although they use a different syntax, Type IV clones carry out the same functionality. Targeted searches based on linguistic and semantic features are made easier by this categorization [178]. The primary objective of clone detection of binary code is to identify and analyze functionally equivalent portions of binary programs. By comparing binary code fragments, the aim is to detect similarities and duplications, which can be valuable for various purposes such as detecting duplicate code, identifying plagiarism, and analyzing malicious code reproduction [179], [77]. In [180], the author introduced an additional approach for detecting identical code by utilizing high-confidence hash techniques and grouping binary programs with matching sections found in the CERT Artifacts Catalogue. On the other hand, studies [181] and [182] focused on performing graph isomorphism (GI) operations on functional pairs of two distinct but comparable binaries. Hence, the limitation of these studies is that GI-based methods can be computationally expensive and may encounter difficulties in scaling when analyzing large or complex binaries. Therefore, Figure 4 illustrates multiple approaches used in clone detection, a critical process in software analysis that aims to identify code clones or duplicated code fragments within a software system.

Furthermore, code changes, such as instruction reordering, can affect the correctness of BinSequence. It also depends on the filtering process, and if the filtration fails to filter out a significant number of functions, the time complexity may increase dramatically. Eventually, binary code-based search engines have also been proposed. The authors in [183] focused on extracting features such as control flow n-grams, but this technique is susceptible to structural changes. The LCS approach is used to align two tracelets in TRACY [184], which disintegrates the CFG into little tracelets. Data can be wasted when CFGs are subdivided into tracelets. Moreover, TRACY is not a good approach when there are fewer than 100 fundamental blocks [185]. Moreover, one common concept in code clone detection is cosine similarity, often used in the metrics-based approach to measure the similarity between

code vectors in equation 2:

$$\text{Sim}(A, B) = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \cdot \sqrt{\sum_{i=1}^{n} B_i^2}}, \tag{2}$$

where $A$ and $B$ are vector representations of code snippets. $A_i$ and $B_i$ are components of the vectors, while $n$ is the number of features in the vector representation. Here, cosine similarity helps quantify the degree of similarity between two code blocks, with values ranging from 0 (completely different) to 1 (identical). This equation is then added to any other method that involves vector-space modeling. Implementation includes calculating dot products of feature vectors, measuring their norms, and normalizing these values to produce similarity scores ranging from 0 to 1.

In the literature, there are five primary models of clone detection: the text-based model is described in [186], the token-based concept is proposed in [187], [188], and [189], the metric-based model is developed in [190], the structure-based model developed, and the behavior-based model [191]. These five modules are mentioned below in detail:

### A. TEXT-BASED APPROACH

Binary code is represented as a series of instruction lines or strings by the text-based method, which is a type of code clone detection. Finding similar lines or strings in the code is the goal of this method. For identifying code snippets that may be compared to textual patterns, this approach works very well [192]. Lexical features are frequently employed in text-based clone detection to detect clones. Specific lexical components, such as variable names, function names, and keywords, are referred to as lexical characteristics in the code. Code snippets that are the same or similar can be identified by comparing these lexical components. This is particularly useful for detecting duplicate code snippets within large code repositories. Jaccard Similarity is used to measure similarity between two sets of tokens:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}, \tag{3}$$

where $A$ and $B$ are sets of tokens from two code snippets. $|A \cap B|$ is the number of common tokens, while $|A \cup B|$ is the total number of unique tokens, in which higher Jaccard similarity means more lexical similarity between two snippets. The process includes tokenizing code fragments, filtering out irrelevant tokens, calculating set intersections and unions, and then determining the ratio to measure similarity. However, the text-based technique does not require transforming the instructions into an intermediate representation (IR), which is an advantage. Unlike existing clone detection techniques that rely on IRs to explain code, the text-based technique operates with the code's text. The primary purpose of the text-based clone detection methods is to locate clones within open-source code repositories [150].

### B. TOKEN-BASED APPROACH

The token-based method is another approach used for identifying code clones, which involves converting code into a sequence of tokens and comparing these tokens to find similar code fragments. This approach converts the code's set of instructions into a list of tokens, where each meaningful part of the code, such as a keyword, operator, identifier, or literal, is recognized as a token. Although the approach claims to use tokens, these tokens must be filtered to eliminate any compiler-induced effects that can distort the results. To put it simply, the filter aims to retrieve the important tokens to lessen the task of recognizing clones while capturing all the noise or irrelevant details.

After filtering, the tokens' level is scrutinized to identify related subsequences. The candidate clone pairs are then marked as such. It is quite easy to determine the relationships between tokens by analyzing the sequences and identifying the presence of certain patterns or identical portions. In the token-based approach, the longest common subsequence (LCS) can be used to measure the similarity between token sequences:

$$LCS(X, Y) = \begin{cases} 0, \\ LCS(X_{n-1}, Y_{m-1}) + 1, \\ \max\left(LCS(X_{n-1}, Y), LCS(X, Y_{m-1})\right), \end{cases} \tag{4}$$

where $X$ and $Y$ are token sequences, $X_n$ and $Y_m$ are the last tokens of each sequence, while LCS finds the longest common subsequence between tokenized representations of code snippets. A dynamic programming method constructs a matrix to record intermediate results, comparing token sequences character by character to determine the optimal alignment length. Apparently, the token-based technique is considered more reliable than the text-based one. It has more reliability due to its starting point, which involves tokens that describe the structural and semantic information of the code. This approach enables easier detection of clones that have been modified through techniques such as code rearrangement, variable renaming, or statement reordering [193].

### C. METRICS-BASED APPROACH

The metrics-based approach is a technique that identifies code clones by encoding blocks of instructions as vectors. This technique converts every block of instructions into a corresponding vector representation, where each vector component represents a specific instruction. The sets of instructions that likely contain duplicate vectors within the codebase are identified by clustering vectors with high similarity. At the same time, the metrics-based method for code clone detection captures blocks of instructions in the form of vectors and clusters them using methods like the nearest neighbor method. This approach facilitates effective distance computing and simplifies the process of locating code clones through the aid of techniques such as LSH. This technique

facilitates the rapid detection of code clones and is used for future research in code clone detection [194].

In the metrics-based approach (vector similarity), the cosine similarity measures the similarity between vectorized representations of code blocks:

$$\text{Sim}(A, B) = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \cdot \sqrt{\sum_{i=1}^{n} B_i^2}}, \tag{5}$$

where $A$ and $B$ are feature vectors representing code blocks. $n$ is the number of features, while the cosine similarity captures structural and functional similarities. The algorithm examines all graph edit operations (insertions, deletions, substitutions) on control flow graphs, employing dynamic programming to determine the least costly sequence of transformations. Within this framework, where each unit of a vector is associated with either an instruction or a specific feature, this method utilizes vector-space modeling to estimate the similarity between code fragments, which is advantageous for large-scale code clone detection due to its efficiency and scalability.

### D. STRUCTURAL-BASED APPROACH

Another method of detecting code clones is made possible through the use of a structural-based approach, which consists of converting the binary code into an assembly file. Basic blocks are blocks of assembly instructions that constitute the assembly file [195]. A structural feature in this case refers to the control flow graph (CFG) of the code. The number of nodes, graph width, graph length, edges, and cyclic complexity are some of these features. These features embody the complexity and structure of the code's control flow. In a structural-based approach, Graph Edit Distance (GED) quantifies the similarity by counting the minimum transformations (insertions, deletions, substitutions) to convert one control flow graph (CFG) to another:

$$GED(G_1, G_2) = \min \sum c(e), \tag{6}$$

where $G_1$ and $G_2$ are two CFGs, $e$ represents an edit operation (node/edge addition, deletion, substitution), $c(e)$ is the cost of edit $e$, and a lower GED indicates more structural similarity. Understanding behavior in structural-based methods involves not only the CFG-based structural components but also other elements linked with the CFG. Implementation utilizes dynamic programming to align behavioral sequences by calculating cumulative distances and identifying optimal warping paths between execution traces. Examples include node coloring, node relationships, and the use of data flow concepts [195]. These concepts, when combined with the CFG, can lead to the extraction of semantic information. Such information stems from the meaning and actions of the constituent parts of code and how they relate to one another. The structural-based technique allows the detection of code clones through structural similarities by using the CFG's structural properties and incorporating other concepts [196].

### E. BEHAVIORAL-BASED APPROACH

In binary analysis, clones refer to situations where two items, such as functions or pieces of code, exhibit similar behavior. Dynamic analysis, which observes how code behaves during runtime, is crucial for identifying these clones. This method is highly effective for locating clones in malicious files. Several dynamic analysis techniques are employed when these files are executed in a sandbox, including monitoring system calls, observing network activity, examining memory usage, and analyzing file actions. In the behavior-based approach, the dynamic time warping (DTW) compares behavioral sequences like (e.g., system calls, execution traces):

$$DTW(X, Y) = \min \sum_{i,j} d\left(X_i, Y_j\right), \tag{7}$$

where $X$ and $Y$ are behavioral sequences, $d\left(X_i, Y_j\right)$ is a distance function (e.g., Euclidean) between two events, while DTW aligns sequences to measure behavioral similarity, effectively detecting malware clones with similar execution behavior. Moreover, runtime data and behaviors are recorded, enabling efficient linking of harmful files for detailed sandbox analysis and improving code clone detection by quantifying similarity in texts, tokens, vectors, and graphs [197].

## V. BINARY AUTHORSHIP ATTRIBUTION

Binary authorship attribution is the process of determining which software developers have made significant contributions to a particular binary program [198], [199]. The goal is to identify developers or teams who contribute more than others, and to determine which developer wrote a binary, which often involves reverse-engineering the code [200]. To assign authorship to specific lines of source code, disassembly and assembler must be used to convert the binary back into source code by following high-level language (HLL) control flow, between functions and variables [201], [202], [203].

These efforts highlight the ongoing exploration of techniques for authorship identification in the context of binary code. Tables 4 and 5 compare systems and tools for authorship attribution research. Table 4 focuses on binary authorship systems, examining features, programming languages, operating systems, and dataset accessibility. The analysis reveals that techniques utilizing all three feature types (syntactic, semantic, and structural) yield the most comprehensive attribution accuracy. Approaches with only two features vary in effectiveness depending on the context. C++ remains the primary language, with Linux being favored for public and GitHub implementations, while Windows-based methods tend to remain private. Public and GitHub implementations are more reproducible and community-validated, though private approaches allow for more controlled experiments. However, Table 5 expands on programming tools, their features, and research applications, offering a comprehensive overview of the field.

Another study [204] introduces a novel approach for detecting multiple authors of binary code by examining the

**TABLE 4.** Comparative analysis of binary authorship attribution techniques.

| Approaches | Features | | | Implementations | | | |
|---|---|---|---|---|---|---|---|
| | Syntactics | Semantics | Structures | Programming languages | Operating Systems | Datasets | Accessibility |
| [24] | ✓ | ✓ | | C++ language | Microsoft Windows | ✓ | Private |
| [207] | ✓ | ✓ | ✓ | C++ language | Linux | ✓ | Public |
| [208] | ✓ | | ✓ | Fortran | Microsoft Windows/Linux | ✓ | GitHub |
| [209] | | ✓ | ✓ | C++ language | Linux | | Private |

**TABLE 5.** Comprehensive comparison of programming tools, features, and research applications.

| Features | Implementations | Programming Languages | Operating Systems | Methodology | Accessibility | References |
|---|---|---|---|---|---|---|
| Syntactics, Semantics | Compiler Optimization | C++ | Microsoft Windows | Abstract Syntax Trees (ASTs), Parsing | Private | https://github.com/microsoft/cppwin32 |
| Syntactics, Semantics, Structures | Static Code Analysis | C++ | Linux | Graph-based Learning, CFGs, AST | Public | https://github.com/IBM/Project_CodeNet |
| Syntactics, Structures | Numerical Analysis | Fortran | Microsoft Windows, Linux | Matrix Computation, Parallelization | GitHub | https://github.com/CUHK-ARISE/ml4code-dataset |
| Semantics, Structures | Simulation Software | C++ | Linux | Event-driven Simulation | Private | https://github.com/github/CodeSearchNet |
| Semantics, Structures | Data Analytics Framework | Python | macOS, Linux | Neural Networks, Graph Analytics | Public | https://arxiv.org/abs/2212.09132 |
| Structures, Syntactics | Enterprise Application Dev | Java | Linux, Windows | Object-Oriented Programming | Commercial | https://huggingface.co/datasets/code-search-net/code_search_net |
| Syntactics, Semantics, AI Extensions | Machine Learning Framework | Python | Cross-platform | Transformer Models, Code Embedding | Open Source | https://ml4code.github.io/ |
| Syntactics, High-level Data Analysis | Statistical Computing Environment | R | Linux, macOS | Statistical Learning, Predictive Modeling | Public | https://arxiv.org/abs/2105.12655 |

authorship of individual basic blocks. This approach extracts basic semantic and syntactic information, including fixed instruction values, backward variable slices, and the structure of function control flow graphs. Additionally, a recent suggestion for authorship characterization is BinChar, put forth by [205]. These endeavors highlight ongoing research in identifying authorship characteristics in the realm of binary code. Furthermore, the binaries were first compiled from C/C++ source code. A training dataset is required for all the methods. In comparison to other approaches, Meng's [206] method requires a smaller dataset. The collection includes CISA C binaries and 22 C++ binaries. Researchers use private, public (accessible but not in public repositories), and general repositories (like GitHub) when possible.

Binary authorship attribution can be categorized into different types based on the characteristics of the binary code being analyzed and the specific context of the attribution task. Here are some common types of binary authorship attribution:

### A. MALWARE AUTHORSHIP ATTRIBUTION

In cybersecurity, malware authorship attribution is an essential field that seeks to identify the people or organizations who produce and disseminate malicious software, including trojans, worms, viruses, and ransomware. Determining the origins of cyberattacks and understanding how different actors strategize, plan, and operate is their main goal [210]. This attribution assists in devising effective security defenses and minimizing the chances of attacks recurring by providing useful information regarding what the attacker's optimal goals are and why they attack in the first place. Identifying the authors of malware involves examining the arrangement of the code, searching for specific characteristics, assessing the frequency of coded phrases, and analyzing the means of dissemination. To predict the possible culprits of future malware cases, machine-learning models may be fed known malware samples with provided author information within the models [211].

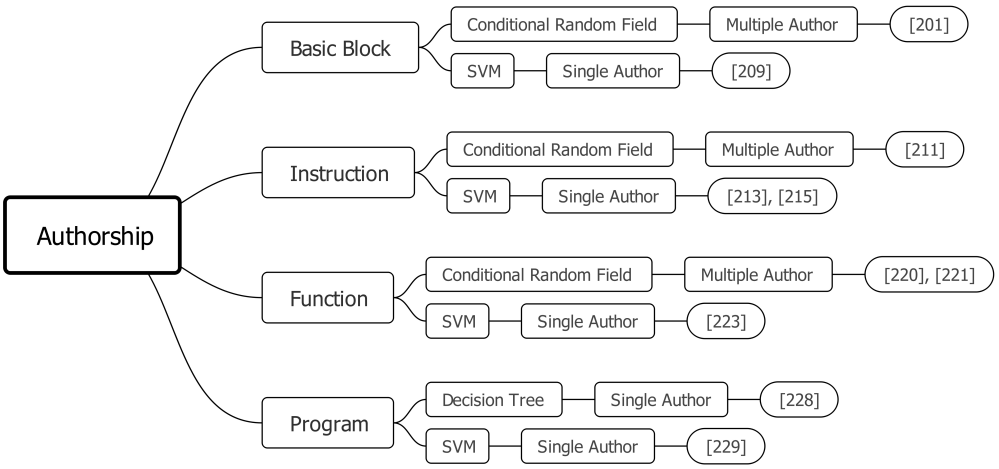



**FIGURE 5.** Taxonomy of binary authorship attribution techniques.

### B. CODE PLAGIARISM DETECTION

The goal of detecting code plagiarism is to find copied or modified code without proper citation. Protecting originality and acknowledging authors, especially in open-source communities, is crucial. Binary authorship attribution helps identify code copying by crediting original developers, aiding enforcement. Attempts to find similarities that are indicative of plagiarism involve comparing binary code samples or built modules. To detect code plagiarism, machine learning models can be trained on labeled datasets and supplemented with tools like MOSS, JPlag, and Codequiry for enhanced accuracy [212], [213].

### C. STYLOMETRIC ATTRIBUTION

Binary authorship attribution can benefit from the application of Stylometric analysis, which is frequently utilized in text-based authorship attribution. In this case, the author of the binary code is identified by analyzing programming ways and coding styles. As shown in Figure 5, it illustrates these programming behaviors, functions, and instructions of various authorship attribution methods (Basic Block, Instruction, Function, Program) using models such as CRF, SVM, and Decision Trees for single or multiple author identification. Therefore, to find distinctive stylistic signatures, characteristics including opcode frequencies, function call patterns, and control flow architectures are analyzed. Stylometric attributes are extracted from the binary code as part of the stylometric attribution procedure. These characteristics may include function call patterns, control flow structures (including loops and conditionals), the frequency and arrangement of opcodes (low-level instructions carried out by the processor), and other traits unique to the code's writing style. When combined, these characteristics create an author's''stylistic signature'' that forms the foundation for identification [214].

Binary authorship attribution may benefit from applying stylometry because it is a method that is notoriously used in text-based authorship attribution. Here, the author of a binary code is identified based on their specific programming habits and particular coding styles. As part of the stylometric attribution process, the attributes are extracted from the binary code. These attributes may be patterns of function calls, structures of control flow like loops and conditionals, the frequencies and arrangements of lower-level instructions supplied by the processor, and attributes subjected to the style of writing the code. The combination of these attributes constructs what is referred to as an author's ''stylistic signature,'' which serves as a basis for individual identification [214].

### D. ATTRIBUTION IN SOFTWARE VULNERABILITIES

Binary authorship attribution is a useful technique for identifying the original authors of software flaws when security flaws are found [215]. Analyzing the susceptible code segments, researching the development history, and examining version control data are methods for assigning blame for software vulnerabilities. To find trends that might lead to future vulnerabilities, machine learning algorithms can be trained on large datasets of known security flaws, such as DiverseVul or PRIMEVUL, enabling proactive vulnerability detection and analysis [216], [217].

## VI. BINARY CODE EVASION AND ANTI-ANALYSIS

Techniques for binary code evasion and anti-analysis are essential in the field of binary code analysis. Attackers use them to evade detection, obstruct analysis, and prevent security researchers, analysts, and malware detection systems from reverse engineering binary code. These strategies are primarily intended to make it difficult for attribution methods to maintain consistent performance across different evasion scenarios [218]. The ongoing improvements in security measures and analysis methods used by defenders require the use of evasion and anti-analysis strategies. Attackers must develop countermeasures to stay ahead and keep their code hidden and effective as security systems and analysts become better at detecting and dissecting malicious code [219].

Adversaries may bypass detection systems based on pattern recognition, thus making analysis more difficult. Another

**TABLE 6.** Overview of Malware analysis evasion and counter-evasion techniques.

| Tools | References | Evasion Technique Mitigated | Methods | Year |
|---|---|---|---|---|
| VMRay Analyzer | https://www.vmray.com/vmray-analyzer/ | Sandbox Detection Evasion | Hypervisor-based Analysis | 2025 |
| Sophos Deep Learning | https://www.sophos.com/en-us/oem/ai-models | Advanced Evasion Techniques | Deep Learning AI Detection | 2025 |
| CISA's Malware Next-Gen | https://www.cisa.gov/resources-tools/services/malware-next-generation-analysis | Advanced Malware Evasion | Automated Malware Analysis | 2024 |
| ANY.RUN's TI Lookup | https://any.run/ | Obfuscation Techniques | Interactive Malware Analysis | 2024 |
| HijackLoader Analysis | https://www.crowdstrike.com/en-us/blog/hijackloader-expands-techniques/ | Hook Bypass Methods | Process Hollowing, Unhooking | 2024 |
| Evasive Malware Techniques | https://nostarch.com/evasive-malware | Various Evasion Techniques | Code Obfuscation, Anti-Debugging | 2024 |
| Angr | https://angr.io | Symbolic Execution | Constraint Solving | 2023 |
| PEiD (PE identifier) | https://www.aldeid.com/wiki/PEiD | Packer Detection | Binary Analysis | 2023 |
| ViperMonkey | https://github.com/decalage2/ViperMonkey | Macro Obfuscation | Office Document Analysis | 2023 |
| PyREBox | https://github.com/Cisco-Talos/pyrebox | VM Detection | Anti-VM Techniques | 2023 |
| Volatility Framework | https://www.volatilityfoundation.org/ | Memory Forensics Evasion | Memory Analysis | 2023 |
| PE-bear | https://github.com/hasherezade/pe-bear | PE File Analysis Evasion | File Analysis | 2023 |
| ScyllaHide | https://github.com/x64dbg/ScyllaHide | Anti-Dumping Techniques | Process Manipulation | 2023 |
| dnSpy | https://github.com/dnSpy/dnSpy | NET Assembly Analysis Evasion | Code DE obfuscation | 2023 |
| Sysmon | http://blog.plura.io/?p=5500 | Evasion of Event Logging | Event Monitoring | 2023 |
| FireEye Malware Analysis Sandbox | https://fireeye.market/apps/219180 | Sandbox Detection | Dynamic Analysis | 2023 |
| ThreatConnect | https://threatconnect.com/ | Threat Intelligence Sharing | Collaborative Analysis | 2023 |
| Falcon Sandbox | https://www.crowdstrike.com/ | Sandbox Detection | Behavioral Analysis | 2023 |
| Hybrid Analysis | https://www.hybrid-analysis.com/ | Automated Malware Analysis | Hybrid Analysis | 2023 |
| Process Monitor | https://learn.microsoft.com/en-us/procmon | Process Monitoring Evasion | Memory Analysis, Dynamic Scanning | 2022 |
| Wireshark | https://www.wireshark.org/ | Network Traffic Anomaly Detection | Traffic Encryption, Protocol Manipulation | 2021 |
| Radare2 | https://github.com/radareorg/radare2 | Packing Detection | DE obfuscation, Static Analysis | 2020 |
| OllyDbg | https://www.ollydbg.de/ | Debugger Detection | Behavior Analysis | 2019 |
| IDA Pro | https://hex-rays.com/ida-pro/ | Code Obfuscation | Static Analysis | 2017 |
| GDB (GNU Debugger) | https://www.sourceware.org/gdb/ | Anti-Debugging Techniques | Runtime Analysis | 2018 |

important element of binary avoidance is anti-debugging. Attackers use code checks to ascertain the presence of any debugging or cybersecurity debugging tools [220], [221].

Table 6 summarizes tools for malware analysis evasion and counter-evasion, aimed at resisting scrutiny by security analysts or defeating malware's evasion tactics. By reviewing the evasion techniques addressed and methods employed, security experts can better understand malware evolution and develop countermeasures.

By employing evasive evasion and anti-analysis strategies, attackers are able to make their code far more complex and sophisticated, which further hides the true nature and purpose of the code. Evasion tactics in the analysis of binary code include numerous methods for altering, obscuring, or hiding the code for detection avoidance [222]. Attackers could employ code obfuscation through encryption, restructuring, and changing variables to make the code complex and difficult to interpret. Such logic makes it even harder for code analysts to identify flaws, and even more, malicious intent. Packing and encryption are other methods of detection avoidance, where the original code is hidden by compressing or encrypting it. These processes turn the code into an obfuscated form, which then needs decryption or unpacking to restore it to its original state [223].

### A. BINARY CODE EVASION TECHNIQUES

In the context of security systems and analysts, evasion strategies involving binary code entail modifying or obscuring the code so that it is not detected by the security systems. These strategies are designed to modify the observed behavior of a particular piece of code, or its signature, so that it becomes more challenging to detect or analyze. Some of the common techniques used for evading detection of binary codes are:

#### 1) CODE OBFUSCATION

One method involves modifying the binary code, making it very difficult to decipher and analyze. Obfuscation techniques include using encryption, reorganizing the code, renaming variables, or adding deceptive code fragments. This requires transforming the binary code, making it harder to examine and understand. The goal is to mislead analysts and obscure the code to make reverse engineering as challenging as possible. Understanding the code's logic and purpose takes time because the intent must be hidden, and the logic designed to achieve it must be complex [224].

When discussing obfuscation, it can be understood as related to code transformations, such as variable renaming or control flow obfuscation. It can be explained as,

$$O_f(x) = f(g(x)), \tag{8}$$

where $f(x)$ is the original code transformation, and $g(x)$ is the obfuscated version to tell how the original code is transformed into a more complex form that makes reverse engineering more difficult.

#### 2) PACKING AND ENCRYPTION

Techniques such as packing and encryption are used to obscure the functionality and content of the binary code, making it accessible for an attacker to pack or encrypt the code using the decryption or unpacking process. Encryption utilizes cryptography to encode and transform binary information into something that cannot be comprehended, while compression packs the code and adds a decompression procedure of sorts. The original code is concealed behind further layers of complexity, which makes it difficult for security systems to discover or analyze the packed or encrypted code [223]. To introduce simple equations for encryption and decryption, such as symmetric key encryption (e.g., AES (Advanced Encryption Standard), which is a symmetric encryption algorithm widely used for securing data).

$$\begin{aligned} E_k(x) &= x \oplus k \text{ (XOR encryption as an example)} \\ D_k\left(E_k(x)\right) &= x \text{ (XOR decryption with key } k), \end{aligned} \tag{9}$$

where $E_k$ represents encryption with key $k$, and $x$ is the data being encrypted, and these two equations demonstrate how encryption hides the code's content and purpose from static analysis.

#### 3) ANTI-DEBUGGING TECHNIQUES

Security researchers employ anti-debugging defense measures to make an analysis harder. The attacker can embed checks into the binary code to detect the presence of a debugger or a breakpoint, which is commonly used during the debugging stage. When a debugger is detected, the code may modify its behavior, which drastically changes how easy or difficult it is to follow the execution flow. In addition, the techniques of self-modification may be applied where the code alters itself whilst being executed in order to obfuscate and complicate the debugging process [225].

### B. ANTI-ANALYSIS TECHNIQUES

Regarding anti-analysis techniques, their aim is to identify and prevent attempts to analyze the binary code. These techniques aim to make it difficult for users to understand the code's functionality, gather relevant data, or comprehend its malicious intent. They are designed to increase the effort and cost required for attack analysis. Here are some common types of anti-analysis techniques:

#### 1) ENVIRONMENT CHECKS

To find out whether an analysis tool, virtualized environment, or sandbox environment is available, which is frequently used by security specialists, attackers perform environment checks on the binary code. These checks may include analyzing specific parts of the registry, system file attributes, or even certain files associated with specific analysis tools. Each scenario contains the possibility of adapting its execution to avoid being examined. Analysts could ease or modify such invasive functionalities, suppress the code to latent conditions, or carry out benign operations. The code can modify its actions to remain undetectable and evade analysis techniques by utilizing behaviorally responsive environment scanning [226].

#### 2) CODE TRIGGERS AND ANTI-VIRTUALIZATION TECHNIQUES

In the case of attackers breaking into a system, there is a need to circumvent triggers put in place to safeguard one's virtual environment from being executed and analyzed. Analysts may use virtualized environments to run potentially harmful code safely, but it remains exposed to several risks. While doing this, their actions become highly relevant because the code can modify its behavioral patterns to evade obfuscation, surveillance, and inspection. The methods often involve attempts to detect hypervisor-specific commands or instructions, verify the existence of specific virtualized device drivers, or identify particular drivers with virtual machines. As such, the code can remain inactive, change the execution flow, or assume a less active state entirely around a virtualized environment [227].

To overcome these barriers, analysts must employ tactics such as dynamic or static methods of analysis, which can be supplemented with emulation or unpacking, or hypervisor introspection, providing a substantial backdrop to anti-virtualization methods. To update their methodologies and mitigate the effects of middle-of-the-road masking predictive methods, analysts need to be prepared quickly. These barriers

can be easily bypassed when an evasion detection system is installed, along with the implementation of static and dynamic analysis. Moreover, to remain updated on emerging anti-analysis techniques, real and active measures need to be taken, with good collaboration and shared expertise among security analysts [228].

#### 3) ANTI-MEMORY FORENSICS TECHNIQUES

In establishing a system's runtime state, an investigation is needed into its memory contents, which is referred to as memory forensics. Memorized information analysis is subject to being obstructed by the use of anti-memory forensics techniques by security attackers. For example, they can store private data in memory in an encrypted format that is geometrically hard to decrypt and decode [229]. Moreover, the use of anti-dumping techniques means that some memory extraction is not possible, thereby complicating the process of obtaining vital information from the memory of the attacked computer for analysts. The so-called anti-analysis and binary code evasion methods demonstrate how sophisticated and determined these attackers are in avoiding scrutiny and circumventing analysis attempts.

To counter these strategies and unveil the underlying binary code, the level of modification of the analysis techniques security researchers and analysts have to use is permanent [230]. With the use of advanced analytic techniques, the implementation and combination of machine learning AI systems, and the utilization of deceptive techniques, analysts are able to enhance their ability to effectively mitigate risks and protect the systems at hand.

### C. CODE INJECTION AND RUNTIME MODIFICATION

Two of the most important methods that an attacker can utilize to shift binary code execution parameters and remain undetected and unanalyzed within the system are code injection and runtime modification. These techniques modify the control flow graph of a program, making it impossible for static analysis tools to reveal the actual purpose of the code. Attackers employ a variety of methods as follows:

#### 1) DYNAMIC CODE GENERATION

The ability to create code dynamically during runtime, which is directly executable in the system memory, is termed dynamic code generation and is frequently used by attackers. This technique's strong claim is necessitated because the generated code does not exist within the parent binary file, therefore making it impossible for static malware analysis tools to identify viruses. Methods for dynamic code generation include the direct use of scripting languages, runtime-compilable libraries, and just-in-time (JIT) compilation. By utilizing dynamic code creation, attackers can conceal their objectives while embedding sophisticated logic that only becomes apparent during program execution. This makes it nearly impossible for analysts to explain the behavior or follow the execution path of the code based on static binary examination. Attackers may change the code structures, load and run modules or functions dynamically, and even create encryption or decryption procedures with dynamic code creation [231].

For dynamic code generation, it can be represented as equations for JIT (Just-in-Time) Compilation: to generate code during runtime, such as the transformation of a high-level language code to machine code dynamically. $C_{\text{runtime}} = JIT(HL)$ where $C_{\text{runtime}}$ is the compiled machine code at runtime, and $HL$ is the high-level code, and its equation describes how $h\downarrow$ level code is compiled dynamically during program execution, making it difficult to detect through static analysis tools.

#### 2) HOOKING AND API INTERCEPTION

Hooking approaches include intercepting and changing system API or function calls inside the binary code. Hackers use hooking to modify input/output parameters or redirect function calls to alternative code paths, thereby influencing the code's behavior. This enables them to carry out harmful operations without directly modifying the binary file, bypassing security checks, or altering the way the code runs. Function hooking libraries, inline function hooking, and system-wide hooking mechanisms are some of the methods for accomplishing hooking. Hooking techniques allow attackers to interfere with the intended execution flow and undermine analytic tools that depend on the behavior of the original code [232].

#### 3) REFLECTIVE DLL INJECTION

An active process can be compromised by inserting a DLL (Dynamic Link Library) using a method called Reflective DLL injection, where the advantages of code injection are not required. Memory space file analysis is evaded with this approach, enabling execution of code as soon as it is loaded. With the ability of Windows to dynamically load DLLs, identifying and analyzing injected code becomes more difficult. This strategy permits evasion of standard scrutiny by malicious functionality modifying a standard operation while hidden as a trusted execution environment member [233].

In a way, runtime modification and code injection raise major concerns for analysts as they offer dynamic features and alter the behavior of the executing code. To tackle these approaches, the analyst will need to employ some form of dynamic analysis techniques during runtime, such as monitoring memory, system call tracing or debugging. By combining static-dynamic analysis, analysts can gain insight into the behavior of the code, unveil the hidden harmful actions, and construct robust responses to create efficient countermeasures in mitigating the associated threats and risks [234].

### D. SELF-MODIFYING CODE AND ENCRYPTION

Both self-modifying code and encryption are utilized to increase the resistance of a binary code to interpretation and

analysis. These processes encrypt sections of the code or change its structure to make reverse engineering attempts difficult. Some of the common techniques in this area are:

#### 1) JUST-IN-TIME (JIT) COMPILATION

In this approach, an attacker begins by gaining the ability to generate new code during program execution through Just-in-Time (JIT) compilation. With JIT compilation, an attacker can change the structure and function of the code by generating it immediately before it needs to be executed. An attacker can use the control flow transformation, variable renaming, or code encryption techniques to make the dynamically produced code more obfuscated.

Such makes static analysis of the code harder to examine and understand. The use of JIT compilation is another way to increase further the level of difficulty that static analysis poses. JIT compilation allows the attacker to take advantage of environmental factors and runtime data to create system-specific hostile code [235].

#### 2) INSTRUCTION ENCRYPTION

The binary code may contain segments that an attacker has embedded and encrypted, making them impossible to understand or run. To carry out the intended task, a decryption process is invoked during program execution, decrypting the encrypted instructions. This method conceals the details of the operations and the purpose of the code, making static analysis more difficult. Without access to the decryption process or decrypted instructions, analysts cannot determine the code's intent or its malicious actions [236].

#### 3) CODE POLYMORPHISM

Polymorphism involves disguising one's identity by altering the appearance of one's code. While maintaining the original purpose, an attacker can develop multiple versions of the code through instruction replacement, reordering, and modifications. Due to these constant changes, such code is hard for static analysis tools to detect and analyze. There are many possible identifiers or patterns to consider, as each code variation is unique. To accurately identify and understand the code's purpose, various examination methods must be used. Examples include pattern recognition and behavior-based analysis.

Attackers can significantly increase the complexity and durability of their malicious code by utilizing these dynamic code generation techniques, which hinder analysis and make it more challenging for security researchers and analysts to identify and eliminate threats. Advanced dynamic analysis techniques and runtime monitoring are crucial for comprehending the behavior and potential risks of dynamically generated code, as static analysis methods alone are often insufficient to reveal the true intentions and capabilities of such code fully.

## VII. OPEN ISSUES AND RESEARCH CHALLENGES

Despite significant progress in binary code analysis and its applications in software security, many critical challenges remain unsolved. One major issue is the absence of standardized approaches, methodologies, and datasets, which greatly limit reproducibility and comparability across different studies. This inconsistency makes it difficult to validate findings from various research groups and hinders the development of a unified knowledge base.

Scalability is another significant challenge, as current methods often require extensive computational resources, rendering them infeasible for analyzing large-scale software systems. The issue is exacerbated by the scarcity of high-quality, labeled ground-truth datasets needed for the effective training and evaluation of machine learning models. Without comprehensive datasets, researchers struggle to develop strong and general solutions.

The ever-changing threat landscape also presents ongoing difficulties, requiring continuous updates to address new vulnerabilities, advanced code obfuscation methods, and evolving evasion tactics. Attackers regularly develop more sophisticated techniques to bypass detection, fueling an ongoing arms race between security experts and malicious actors. Privacy and legal issues pose significant barriers when analyzing proprietary or sensitive software, restricting the scope of research and real-world applications. Additionally, current techniques face major challenges in accurately attributing authorship and handling code that is dynamically generated or heavily obfuscated.

A core limitation of current methods is their inability to analyze context. Most analyze code segments in isolation, neglecting the broader interactions between code components and their operating environment. This weakness reduces the effectiveness of vulnerability detection and similarity analysis in complex, real-world software systems. Moreover, existing machine learning approaches often lack resilience against adversarial attacks, rendering them susceptible to advanced evasion techniques employed by modern malware. The lack of standardized evaluation benchmarks also complicates the comparison and fair measurement of different methods.

Additionally, our survey acknowledges a scope limitation regarding context-aware analysis techniques, a deliberate boundary established to maintain focus on foundational binary analysis methods. While this survey comprehensively covers six key areas (fingerprinting, vulnerability detection, similarity analysis, authorship attribution, clone identification, and anti-evasion methods), it primarily examines traditional approaches that analyze code segments in isolation. Still, we recognize that current and rapidly advancing techniques analyzing operational memory layouts, environmental conditions, and interactions between components are an important and complementary area for further research and practical enhancement.

## VIII. CONCLUSION AND FUTURE WORK

In conclusion, this comprehensive review has underscored the vital role of binary code analysis in ensuring the security and reliability of software systems. The review plays a pivotal role in enhancing the security, reliability, and functionality of modern software systems, especially in situations where source code is not accessible. This review has critically examined current techniques across six core areas, highlighting their strengths, limitations, and practical applications. By categorizing existing methods, introducing taxonomies, and identifying key challenges in the field, such as similarity detection, attribution accuracy, and scalability, this study provides a comprehensive foundation for both researchers and practitioners. This review thus serves as a reference point and roadmap for ongoing research and practical developments in binary code analysis.

This research is crucial for ensuring security tools remain effective against constantly changing threats. Creating standardized benchmarks with high-quality, labeled datasets is essential for enhancing reproducibility and facilitating fair comparisons between different methods. Such standardization will promote better collaboration among academics and industry professionals.

To address current challenges, future research should focus on developing AI-driven techniques that create more robust and scalable analytical frameworks. Graph-based deep learning models are particularly promising for understanding complex structural relationships in binary code, thereby enhancing both similarity detection and authorship attribution accuracy. Contrastive learning methods hold great potential for strengthening binary code representation, particularly in identifying obfuscated and polymorphic malware. To manage scalability, researchers should explore distributed and federated learning approaches that support large-scale analysis without compromising efficiency or accuracy. Incorporating adversarial robustness into machine learning-based binary analysis tools is vital for resisting sophisticated obfuscation and evasion strategies.

Future work should focus on hybrid static-dynamic analysis techniques that provide comprehensive context and behavior insights, greatly improving vulnerability detection in complex software. Key priorities include enhancing binary code similarity algorithms by merging structural and semantic analysis, developing advanced fingerprinting techniques for improved scalability and accuracy, utilizing AI for automated vulnerability detection, improving clone detection amid obfuscation, studying stylistic and behavioral patterns for authorship attribution, and developing countermeasures against anti-analysis techniques. Incorporating contextual analysis into binary code methods offers deeper understanding of code relationships and vulnerabilities. Advances in algorithms and technology will improve scalability, accuracy, and adaptation to new threats. Addressing these challenges and exploring innovative research can significantly improve software security, vulnerability detection, and defenses against sophisticated attacks. This evolving field aims to deliver scalable, reliable solutions for practical security needs, strengthening our digital infrastructure against emerging threats.

## REFERENCES

[1] L. Di Grazia and M. Pradel, "Code search: A survey of techniques for finding code," *ACM Comput. Surv.*, vol. 55, no. 11, pp. 1–31, Nov. 2023, doi: 10.1145/3565971.

[2] I. U. Haq and J. Caballero, "A survey of binary code similarity," *ACM Comput. Surv.*, vol. 54, no. 3, pp. 1–38, Apr. 2022, doi: 10.1145/3446371.

[3] A. Jia, M. Fan, W. Jin, X. Xu, Z. Zhou, Q. Tang, S. Nie, S. Wu, and T. Liu, "1-to-1 or 1-to-n? Investigating the effect of function inlining on binary similarity analysis," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 4, pp. 1–26, Oct. 2023, doi: 10.1145/3561385.

[4] K.-H. Liu, J. Gao, Y. Xu, K.-J. Feng, X.-N. Ye, S.-T. Liong, and L.-Y. Chen, "A novel soft-coded error-correcting output codes algorithm," *Pattern Recognit.*, vol. 134, Feb. 2023, Art. no. 109122, doi: 10.1016/j.patcog.2022.109122.

[5] Y. Younan, W. Joosen, and F. Piessens, "Runtime countermeasures for code injection attacks against C and C++ programs," *ACM Comput. Surv.*, vol. 44, no. 3, pp. 1–28, Jun. 2012, doi: 10.1145/2187671.2187679.

[6] M. M. Rahman and C. K. Roy, "A systematic review of automated query reformulations in source code search," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 6, pp. 1–79, Nov. 2023, doi: 10.1145/3607179.

[7] H. Patel, S. Guttula, N. Gupta, S. Hans, R. S. Mittal, and N. Lokesh, "A data-centric AI framework for automating exploratory data analysis and data quality tasks," *J. Data Inf. Quality*, vol. 15, no. 4, pp. 1–26, Dec. 2023, doi: 10.1145/3603709.

[8] P. Zhang, C. Wu, M. Peng, K. Zeng, D. Yu, Y. Lai, Y. Kang, W. Wang, and Z. Wang, "Khaos: The impact of inter-procedural code obfuscation on binary diffing techniques," in *Proc. 21st ACM/IEEE Int. Symp. Code Gener. Optim.*, Feb. 2023, pp. 55–67, doi: 10.1145/3579990.3580007.

[9] D. Patel, T. Rajesh, and G. Balamurugan, "Enhancing cybersecurity vigilance with deep learning for malware detection," in *Proc. 10th Int. Conf. Commun. Signal Process. (ICCSP)*, Apr. 2024, pp. 1005–1010, doi: 10.1109/ICCSP60870.2024.10544228.

[10] S. Marksteiner, P. Priller, and M. Wolf, "Approaches for automating cybersecurity testing of connected vehicles," in *Intelligent Secure Trustable Things* (Studies in Computational Intelligence). Cham, Switzerland: Springer, 2024, pp. 219–234, doi: 10.1007/978-3-031-54049-3_13.

[11] L. Wang, Y. Zheng, D. Jin, F. Li, Y. Qiao, and S. Pan, "Contrastive graph similarity networks," *ACM Trans. Web*, vol. 18, no. 2, pp. 1–20, May 2024, doi: 10.1145/3580511.

[12] W. Mu, Y. Zhang, B. Huang, J. Guo, and S. Cui, "A hotspot-driven semi-automated competitive analysis framework for identifying compiler key optimizations," in *Proc. 32nd ACM SIGPLAN Int. Conf. Compiler Construct.*, Feb. 2023, pp. 216–227, doi: 10.1145/3578360.3580255.

[13] M. Beninger, P. Charland, S. H. H. Ding, and B. C. M. Fung, "ERS0: Enhancing military cybersecurity with AI-driven SBOM for firmware vulnerability detection and asset management," in *Proc. 16th Int. Conf. Cyber Conflict, Over Horiz. (CyCon)*, May 2024, pp. 141–160, doi: 10.23919/CyCon62501.2024.10685598.

[14] A. Adhikari and P. Kulkarni, "Survey of techniques to detect common weaknesses in program binaries," *Cyber Secur. Appl.*, vol. 3, Dec. 2025, Art. no. 100061, doi: 10.1016/j.csa.2024.100061.

[15] F. Deldar and M. Abadi, "Deep learning for zero-day malware detection and classification: A survey," *ACM Comput. Surv.*, vol. 56, no. 2, pp. 1–37, Feb. 2024, doi: 10.1145/3605775.

[16] S. Lee, J. Lee, J. Ko, and J. Shim, "Crosys: Cross architectural dynamic analysis," in *Proc. 12th ACM SIGPLAN Int. Workshop State Art Program Anal.*, Jun. 2023, pp. 55–62, doi: 10.1145/3589250.3596147.

[17] Z. Huang, S. Song, H. Liu, H. Kuang, J. Zhang, and P. Hu, "A review on binary code analysis datasets," in *Proc. Int. Conf. Wireless Artif. Intell. Comput. Syst. Appl.*, 2024, pp. 199–214, doi: 10.1007/978-3-031-71470-2_17.

[18] S. M. Rao and A. Jain, "Advances in malware analysis and detection in cloud computing environments: A review," *Int. J. Saf. Secur. Eng.*, vol. 14, no. 1, pp. 225–230, Feb. 2024.

[19] J. Ferdous, R. Islam, A. Mahboubi, and M. Zahidul Islam, "AI-based ransomware detection: A comprehensive review," *IEEE Access*, vol. 12, pp. 136666–136695, 2024.

[20] S. Yang, Z. Xu, Y. Xiao, Z. Lang, W. Tang, Y. Liu, Z. Shi, H. Li, and L. Sun, "Towards practical binary code similarity detection: Vulnerability verification via patch semantic analysis," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 6, pp. 1–29, Nov. 2023, doi: 10.1145/3604608.

[21] G. Wu and H. Tang, "Binary code vulnerability detection based on multi-level feature fusion," *IEEE Access*, vol. 11, pp. 63904–63915, 2023, doi: 10.1109/ACCESS.2023.3289001.

[22] L. F. G. Gutiérrez and B. Keith, "A systematic literature review on word embeddings," in *Proc. Int. Conf. Softw. Process Improvement*, 2018, pp. 132–141.

[23] X. Wu and H. Wang, "Asymmetric contrastive learning for audio fingerprinting," *IEEE Signal Process. Lett.*, vol. 29, pp. 1873–1877, 2022, doi: 10.1109/LSP.2022.3201430.

[24] M. Zakeri-Nasrabadi, S. Parsa, M. Ramezani, C. Roy, and M. Ekhtiarzadeh, "A systematic literature review on source code similarity measurement and clone detection: Techniques, applications, and challenges," *J. Syst. Softw.*, vol. 204, Oct. 2023, Art. no. 111796, doi: 10.1016/j.jss.2023.111796.

[25] Z. Chen and M. Monperrus, "A literature study of embeddings on source code," 2019, *arXiv:1904.03061*.

[26] P. Xu, Z. Mai, Y. Lin, Z. Guo, and V. S. Sheng, "A survey on binary code vulnerability mining technology," *J. Inf. Hiding Privacy Protection*, vol. 3, no. 4, pp. 165–179, 2021, doi: 10.32604/jihpp.2021.027280.

[27] X. Feng, X. Zhu, Q.-L. Han, W. Zhou, S. Wen, and Y. Xiang, "Detecting vulnerability on IoT device firmware: A survey," *IEEE/CAA J. Autom. Sinica*, vol. 10, no. 1, pp. 25–41, Jan. 2023, doi: 10.1109/JAS.2022.105860.

[28] S. Alrabaee, M. Debbabi, and L. Wang, "A survey of binary code fingerprinting approaches: Taxonomy, methodologies, and features," *ACM Comput. Surv.*, vol. 55, no. 1, pp. 1–41, Jan. 2023, doi: 10.1145/3486860.

[29] H. Zhang and K. Sakurai, "A survey of software clone detection from security perspective," *IEEE Access*, vol. 9, pp. 48157–48173, 2021, doi: 10.1109/ACCESS.2021.3065872.

[30] A. S. Filho, R. J. Rodríguez, and E. L. Feitosa, "Evasion and countermeasures techniques to detect dynamic binary instrumentation frameworks," *Digit. Threats, Res. Pract.*, vol. 3, no. 2, pp. 1–28, Jun. 2022, doi: 10.1145/3480463.

[31] T. Alsmadi and N. Alqudah, "A survey on malware detection techniques," in *Proc. Int. Conf. Inf. Technol. (ICIT)*, Jul. 2021, pp. 371–376, doi: 10.1109/ICIT52682.2021.9491765.

[32] H. Wang, M. J. Bah, and M. Hammad, "Progress in outlier detection techniques: A survey," *IEEE Access*, vol. 7, pp. 107964–108000, 2019, doi: 10.1109/ACCESS.2019.2932769.

[33] V. Kalgutkar, R. Kaur, H. Gonzalez, N. Stakhanova, and A. Matyukhina, "Code authorship attribution: Methods and challenges," *ACM Comput. Surv.*, vol. 52, no. 1, pp. 1–36, Jan. 2020, doi: 10.1145/3292577.

[34] O. Or-Meir, N. Nissim, Y. Elovici, and L. Rokach, "Dynamic malware analysis in the modern era—A state of the art survey," *ACM Comput. Surv.*, vol. 52, no. 5, pp. 1–48, Sep. 2020, doi: 10.1145/3329786.

[35] A. Souri and R. Hosseini, "A state-of-the-art survey of malware detection approaches using data mining techniques," *Human-Centric Comput. Inf. Sci.*, vol. 8, no. 1, p. 3, Dec. 2018, doi: 10.1186/s13673-018-0125-x.

[36] E. Masabo, K. Swaib Kaawaase, J. Sansa-Otim, J. Ngubiri, and D. Hanyurwimfura, "A state of the art survey on polymorphic malware analysis and detection techniques a state of the art survey on polymorphic malware analysis and detection techniques," *ICATACT J.*, vol. 8, pp. 1–13, Aug. 2018.

[37] L. Simko, L. Zettlemoyer, and T. Kohno, "Recognizing and imitating programmer style: Adversaries in program authorship attribution," *Proc. Privacy Enhancing Technol.*, vol. 2018, no. 1, pp. 127–144, Jan. 2018, doi: 10.1515/popets-2018-0007.

[38] A. Afianian, S. Niksefat, B. Sadeghiyan, and D. Baptiste, "Malware dynamic analysis evasion techniques: A survey," *ACM Comput. Surv.*, vol. 52, no. 6, pp. 1–28, Nov. 2020, doi: 10.1145/3365001.

[39] S. Wang and D. Wu, "In-memory fuzzing for binary code similarity analysis," in *Proc. 32nd IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Oct. 2017, pp. 319–330, doi: 10.1109/ASE.2017.8115645.

[40] M. White, M. Tufano, C. Vendome, and D. Poshyvanyk, "Deep learning code fragments for code clone detection," in *Proc. 31st IEEE/ACM Int. Conf. Automated Softw. Eng.*, Aug. 2016, pp. 87–98, doi: 10.1145/2970276.2970326.

[41] V. Guruswami, "List decoding of binary codes—A brief survey of some recent results," in *Proc. Int. Conf. Coding Cryptol.*, 2009, pp. 97–106, doi: 10.1007/978-3-642-01877-0_10.

[42] G. Chatley, S. Kaur, and B. Sohal, "Software clone detection: A review," *Int. J. Control Theory Appl.*, vol. 9, no. 41, pp. 555–563, 2016.

[43] E. Stamatatos, "A survey of modern authorship attribution methods," *J. Amer. Soc. Inf. Sci. Technol.*, vol. 60, no. 3, pp. 538–556, Mar. 2009, doi: 10.1002/asi.21001.

[44] C. K. Roy and J. R. Cordy, "A survey on software clone detection research," *Queen's Sch. Comput. TR*, vol. 541, no. 115, p. 115, Feb. 2007. [Online]. Available: http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.62.7869%5Cnhttp://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.67.9931&rep=rep1&type=pdf

[45] V. Chandola, A. Banerjee, and V. Kumar, "Outlier detection: A survey," *ACM Comput. Surv.*, vol. 14, p. 15, Mar. 2007.

[46] D. Binkley, "Source code analysis: A road map," in *Proc. Future Softw. Eng. (FOSE)*, May 2007, pp. 104–119, doi: 10.1109/FOSE.2007.27.

[47] X. Zhang, J. Heaps, R. Slavin, J. Niu, T. Breaux, and X. Wang, "DAISY: Dynamic-analysis-induced source discovery for sensitive data," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 4, pp. 1–34, Oct. 2023, doi: 10.1145/3569936.

[48] J. Senanayake, H. Kalutarage, M. O. Al-Kadri, A. Petrovski, and L. Piras, "Android source code vulnerability detection: A systematic literature review," *ACM Comput. Surv.*, vol. 55, no. 9, pp. 1–37, Sep. 2023, doi: 10.1145/3556974.

[49] I. Yoo, "Visualizing windows executable viruses using self-organizing maps," in *Proc. ACM workshop Visualizat. data mining Comput. Secur.*, Oct. 2004, pp. 82–89, doi: 10.1145/1029208.1029222.

[50] M. Cen, X. Deng, F. Jiang, and R. Doss, "Zero-ran sniff: A zero-day ransomware early detection method based on zero-shot learning," *Comput. Secur.*, vol. 142, Jul. 2024, Art. no. 103849.

[51] M. Brohet and F. Regazzoni, "A survey on thwarting memory corruption in RISC-V," *ACM Comput. Surv.*, vol. 56, no. 2, pp. 1–29, Feb. 2024, doi: 10.1145/3604906.

[52] A. Costea, A. Tiwari, S. Chianasta, R. Kishore, A. Roychoudhury, and I. Sergey, "Hippodrome: Data race repair using static analysis summaries," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 2, pp. 1–33, Mar. 2023, doi: 10.1145/3546942.

[53] H. Mousavi, A. Ebnenasir, and E. Mahmoudzadeh, "Formal specification, verification and repair of Contiki's scheduler," *ACM Trans. Cyber-Phys. Syst.*, vol. 7, no. 4, pp. 1–28, Oct. 2023, doi: 10.1145/3605948.

[54] M. Mansouri, J. Xu, and G. Portokalidis, "Eliminating vulnerabilities by disabling unwanted functionality in binary programs," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, Jul. 2023, pp. 259–273, doi: 10.1145/3579856.3595796.

[55] S. Taviss, S. H. H. Ding, M. Zulkernine, P. Charland, and S. Acharya, "Asm2Seq: Explainable assembly code functional summary generation for reverse engineering and vulnerability analysis," *Digit. Threats, Res. Pract.*, vol. 5, no. 1, pp. 1–25, Mar. 2024, doi: 10.1145/3592623.

[56] N. Ivanov, C. Li, Q. Yan, Z. Sun, Z. Cao, and X. Luo, "Security threat mitigation for smart contracts: A comprehensive survey," *ACM Comput. Surv.*, vol. 55, no. 14s, pp. 1–37, Dec. 2023, doi: 10.1145/3593293.

[57] B. Pan, N. Stakhanova, and S. Ray, "Data provenance in security and privacy," *ACM Comput. Surv.*, vol. 55, no. 14s, pp. 1–35, Dec. 2023, doi: 10.1145/3593294.

[58] T. Wang, H. Cheng, K. P. Chow, and L. Nie, "Deep convolutional pooling transformer for deepfake detection," *ACM Trans. Multimedia Comput., Commun., Appl.*, vol. 19, no. 6, pp. 1–20, Nov. 2023, doi: 10.1145/3588574.

[59] P. Bajpai and R. Enbody, "Know thy ransomware response: A detailed framework for devising effective ransomware response strategies," *Digit. Threats: Res. Pract.*, vol. 4, no. 4, pp. 1–19, Dec. 2023, doi: 10.1145/3606022.

[60] D. K. Mahto, A. K. Singh, K. N. Singh, O. P. Singh, and A. K. Agrawal, "Robust copyright protection technique with high-embedding capacity for color images," *ACM Trans. Multimedia Comput., Commun., Appl.*, vol. 20, no. 11, pp. 1–12, Nov. 2024, doi: 10.1145/3580502.

[61] F. Ma, M. Ren, L. Ouyang, Y. Chen, J. Zhu, T. Chen, Y. Zheng, X. Dai, Y. Jiang, and J. Sun, "Pied-piper: Revealing the backdoor threats in Ethereum ERC token contracts," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 3, pp. 1–24, Jul. 2023, doi: 10.1145/3560264.

[62] D. Larraz, A. Viswanathan, C. Tinelli, and M. Laurent, "Beyond model checking of idealized lustre in kind 2," *ACM SIGAda Ada Lett.*, vol. 42, no. 2, pp. 44–48, Apr. 2023, doi: 10.1145/3591335.3591338.

[63] C. Groh, S. Proskurin, and A. Zarras, "Free willy: Prune system calls to enhance software security," in *Proc. 38th ACM/SIGAPP Symp. Appl. Comput.*, Mar. 2023, pp. 1522–1529, doi: 10.1145/3555776.3577593.

[64] L. Zhou, X. Bai, X. Liu, J. Zhou, and E. R. Hancock, "Learning binary code for fast nearest subspace search," *Pattern Recognit.*, vol. 98, Feb. 2020, Art. no. 107040, doi: 10.1016/j.patcog.2019.107040.

[65] S. Devkota, P. Aschwanden, A. Kunen, M. Legendre, and K. E. Isaacs, "CcNav: Understanding compiler optimizations in binary code," *IEEE Trans. Vis. Comput. Graph.*, vol. 27, no. 2, pp. 667–677, Feb. 2021, doi: 10.1109/TVCG.2020.3030357.

[66] B. Liu, W. Huo, C. Zhang, W. Li, F. Li, A. Piao, and W. Zou, "$\alpha$Diff: Cross-version binary code similarity detection with DNN," in *Proc. 33rd IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Sep. 2018, pp. 667–678, doi: 10.1145/3238147.3238199.

[67] Y. Hu, Y. Zhang, J. Li, and D. Gu, "Binary code clone detection across architectures and compiling configurations," in *Proc. IEEE/ACM 25th Int. Conf. Program Comprehension (ICPC)*, May 2017, pp. 88–98, doi: 10.1109/ICPC.2017.22.

[68] P. Sun, L. Garcia, G. Salles-Loustau, and S. Zonouz, "Hybrid firmware analysis for known mobile and IoT security vulnerabilities," in *Proc. 50th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, Jun. 2020, pp. 373–384, doi: 10.1109/DSN48063.2020.00053.

[69] F. Fowze, D. Tian, G. Hernandez, K. Butler, and T. Yavuz, "ProX-ray: Protocol model learning and guided firmware analysis," *IEEE Trans. Softw. Eng.*, vol. 47, no. 9, pp. 1907–1928, Sep. 2021, doi: 10.1109/TSE.2019.2939526.

[70] G. Hernandez, F. Fowze, D. J. Tian, T. Yavuz, P. Traynor, and K. R. B. Butler, "Toward automated firmware analysis in the IoT era," *IEEE Secur. Privacy*, vol. 17, no. 5, pp. 38–46, Sep. 2019, doi: 10.1109/MSEC.2019.2926462.

[71] D. D. Chen, M. Egele, M. Woo, and D. Brumley, "Towards automated dynamic analysis for Linux-based embedded firmware," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2016, pp. 1–11, doi: 10.14722/ndss.2016.23415.

[72] M. Bettayeb, Q. Nasir, and M. A. Talib, "Firmware update attacks and security for IoT devices: Survey," in *Proc. ArabWIC 6th Annu. Int. Conf. Res. Track*, Mar. 2019, pp. 1–6, doi: 10.1145/3333165.3333169.

[73] H. Liu, R. Ji, Y. Wu, F. Huang, and B. Zhang, "Cross-modality binary code learning via fusion similarity hashing," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 6345–6353, doi: 10.1109/CVPR.2017.672.

[74] C. Oprisa, G. Cabau, and G. S. Pal, "A new string distance for computing binary code similarities," in *Proc. IEEE 10th Int. Conf. Intell. Comput. Commun. Process. (ICCP)*, Sep. 2014, pp. 91–96, doi: 10.1109/ICCP.2014.6936986.

[75] F. Hoffmeister and T. Bäck, "Genetic algorithms and evolution strategies: Similarities and differences," in *Proc. Int. Conf. Parallel Problem Solving Nature*, 2006, pp. 455–469, doi: 10.1007/bfb0029787.

[76] J. Pewny, B. Garmany, R. Gawlik, C. Rossow, and T. Holz, "Cross-architecture bug search in binary executables," in *Proc. IEEE Symp. Secur. Privacy*, May 2015, pp. 83–91, doi: 10.1109/SP.2015.49.

[77] L. Luo, J. Ming, D. Wu, P. Liu, and S. Zhu, "Semantics-based obfuscation-resilient binary code similarity comparison with applications to software plagiarism detection," in *Proc. 22nd ACM SIGSOFT Int. Symp. Found. Softw. Eng.*, Nov. 2014, pp. 389–400, doi: 10.1145/2635868.2635900.

[78] F. A. Fellouse, B. Li, D. M. Compaan, A. A. Peden, S. G. Hymowitz, and S. S. Sidhu, "Molecular recognition by a binary code," *J. Mol. Biol.*, vol. 348, no. 5, pp. 1153–1162, May 2005, doi: 10.1016/j.jmb.2005.03.041.

[79] Z. Liu, "Binary code similarity detection," in *Proc. 36th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Nov. 2021, pp. 1056–1060, doi: 10.1109/ASE51524.2021.9678518.

[80] S. Alrabaee, M. Debbabi, P. Shirani, L. Wang, A. Youssef, A. Rahimian, L. Nouh, D. Mouheb, H. Huang, and A. Hanna, "Binary analysis overview," in *Binary Code Fingerprinting for Cybersecurity: Application to Malicious Code Fingerprinting* (Advances in Information Security), vol. 78. Cham, Switzerland: Springer, 2020, pp. 7–44, doi: 10.1007/978-3-030-34238-8_2.

[81] L.-A. Daniel, "Symbolic binary-level code analysis for security," Ph.D. dissertation, Univ. Côte d'Azur, Nice, France, 2021.

[82] F. Shen, X. Zhou, Y. Yang, J. Song, H. T. Shen, and D. Tao, "A fast optimization method for general binary code learning," *IEEE Trans. Image Process.*, vol. 25, no. 12, pp. 5610–5621, Dec. 2016, doi: 10.1109/TIP.2016.2612883.

[83] L.-A. Daniel, "Symbolic binary-level code analysis for security. Application to the detection of microarchitectural timing attacks in cryptographic code," Université Côte d'Azur, Nice, France, Tech. Rep., 2021.

[84] A. Rahimian, L. Nouh, D. Mouheb, and H. Huang, *Binary Code Fingerprinting for Cybersecurity: Application to Malicious Code Fingerprinting*. Cham, Switzerland: Springer, 2020, p. 264. [Online]. Available: https://link.springer.com/content/pdf/10.1007/978-3-030-34238-8.pdf

[85] K. Redmond, "An instruction embedding model for binary code analysis," M.S. thesis, Dept. South Carolina, University of South Carolina, Columbia, SC, USA, 2019. [Online]. Available: https://scholarcommons.sc.edu/etd/5299

[86] Z. Yu, R. Cao, Q. Tang, S. Nie, J. Huang, and S. Wu, "Order matters: Semantic-aware neural networks for binary code similarity detection," in *Proc. AAAI - 34th AAAI Conf. Artif. Intell.*, vol. 34, 2020, pp. 1145–1152, doi: 10.1609/aaai.v34i01.5466.

[87] M. Qiao, X. Zhang, H. Sun, Z. Shan, F. Liu, W. Sun, and X. Li, "Multi-level cross-architecture binary code similarity metric," *Arabian J. Sci. Eng.*, vol. 46, no. 9, pp. 8603–8615, Sep. 2021, doi: 10.1007/s13369-021-05630-7.

[88] Q. Wang, B. Shen, S. Wang, L. Li, and L. Si, "Binary codes embedding for fast image tagging with incomplete labels," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 425–439, doi: 10.1007/978-3-319-10605-2_28.

[89] A. Sheikh, A. G. I. Amat, and G. Liva, "Binary message passing decoding of product codes based on generalized minimum distance decoding : (Invited paper)," in *Proc. 53rd Annu. Conf. Inf. Sci. Syst. (CISS)*, Mar. 2019, pp. 1–5, doi: 10.1109/CISS.2019.8692862.

[90] L. Nouh, A. Rahimian, D. Mouheb, M. Debbabi, and A. Hanna, "BinSign: Fingerprinting binary functions to support automated analysis of code executables," in *Proc. IFIP Int. Conf. ICT Syst. Secur. Privacy Protection*, 2017, pp. 341–355, doi: 10.1007/978-3-319-58469-0_23.

[91] Y. Li, H. Zhao, Z. Cao, E. Liu, and L. Pang, "Compact and cancelable fingerprint binary codes generation via one permutation hashing," *IEEE Signal Process. Lett.*, vol. 28, pp. 738–742, 2021, doi: 10.1109/LSP.2021.3071262.

[92] J. Fan, Y. Gu, M. Hachimori, and Y. Miao, "Signature codes for weighted binary adder channel and multimedia fingerprinting," *IEEE Trans. Inf. Theory*, vol. 67, no. 1, pp. 200–216, Jan. 2021, doi: 10.1109/TIT.2020.3033445.

[93] W.-G. Kim and Y. Suh, "Short N-secure fingerprinting code for image," in *Proc. Int. Conf. Image Process. (ICIP)*, vol. 4, Apr. 2004, pp. 2167–2170, doi: 10.1109/ICIP.2004.1421525.

[94] I. Keivanloo, C. K. Roy, and J. Rilling, "Java bytecode clone detection via relaxation on code fingerprint and semantic Web reasoning," in *Proc. 6th Int. Workshop Softw. Clones (IWSC)*, Jun. 2012, pp. 36–42, doi: 10.1109/IWSC.2012.6227864.

[95] J. Kim, S.-W. Lee, D. Cho, and J. M. Youn, "Malware visualization and similarity via tracking binary execution path," *Teh. Vjesn.*, vol. 29, no. 1, pp. 221–230, 2021, doi: 10.17559/tv-20210820065715.

[96] E. Cozzi, M. Graziano, Y. Fratantonio, and D. Balzarotti, "Understanding linux malware," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2018, pp. 161–175, doi: 10.1109/SP.2018.00054.

[97] S. Alrabaee, M. Debbabi, and L. Wang, "On the feasibility of binary authorship characterization," *Digit. Invest.*, vol. 28, pp. S3–S11, Apr. 2019, doi: 10.1016/j.diin.2019.01.028.

[98] A. N. Moran and J. T. Bennett, *Supply Chain Analysis: From Quartermaster to Sunshop*, vol. 11. Milpitas, CA, USA: FireEye, 2013.

[99] A. Caliskan-Islam, R. Harang, A. Liu, A. Narayanan, C. R. Voss, F. Yamaguchi, and R. Greenstadt, "De-anonymizing programmers via code stylometry," in *Proc. 24th USENIX Secur. Symp.*, 2015, pp. 255–270.

[100] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel, and G. Vigna, "SOK: (State of) the art of war: Offensive techniques in binary analysis," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2016, pp. 138–157, doi: 10.1109/SP.2016.17.

[101] N. E. Rosenblum, B. P. Miller, and X. Zhu, "Extracting compiler provenance from program binaries," in *Proc. 9th ACM SIGPLAN-SIGSOFT workshop Program Anal. Softw. tools Eng.*, May 2010, pp. 21–28, doi: 10.1145/1806672.1806678.

[102] E. C. R. Shin, D. Song, and R. Moazzezi, "Recognizing functions in binaries with neural networks," in *Proc. 24th USENIX Secur. Symp.*, 2015, pp. 611–626.

[103] P. Liu, L. He, and Z. Li, "A survey on deep learning for website fingerprinting attacks and defenses," *IEEE Access*, vol. 11, pp. 26033–26047, 2023, doi: 10.1109/ACCESS.2023.3253559.

[104] B. Rodrigues, E. J. Scheid, K. O. E. Müller, J. Willems, and B. Stiller, "Real-time tracking of medical devices: An analysis of multilateration and fingerprinting approaches," 2023, *arXiv:2303.01151*.

[105] J. Kinder, "Static analysis of x86 executables," Ph.D. dissertation, Technische Univ. Darmstadt, Darmstadt, Germany, 2010. [Online]. Available: http://www.cs.rhul.ac.uk/home/kinder/papers/phdthesis.pdf

[106] M. Zhou, X. Gao, J. Wu, J. Grundy, X. Chen, C. Chen, and L. Li, "ModelObfuscator: Obfuscating model information to protect deployed ML-based systems," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2023, pp. 1005–1017, doi: 10.1145/3597926.3598113.

[107] M. Egele, T. Scholte, E. Kirda, and C. Kruegel, "A survey on automated dynamic malware-analysis techniques and tools," *ACM Comput. Surv.*, vol. 44, no. 2, pp. 1–42, Feb. 2012, doi: 10.1145/2089125.2089126.

[108] K. Roundy and B. P. Miller, "Hybrid analysis and control of malware," in *Proc. Int. Workshop Recent Adv. Intrusion Detection*, 2010, pp. 317–338, doi: 10.1007/978-3-642-15512-3_17.

[109] K. Zhang, Z. Liu, J. Hu, and S. Wang, "An auto-encoder based method for camera fingerprint compression," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Jun. 2023, pp. 1–5, doi: 10.1109/ICASSP49357.2023.10094951.

[110] A. Rahimian, P. Shirani, S. Alrbaee, L. Wang, and M. Debbabi, "BinComp: A stratified approach to compiler provenance attribution," in *Proc. Digit. Forensic Res. Conf. DFRWS USA*, vol. 14, 2015, pp. S146–S155, doi: 10.1016/j.diin.2015.05.015.

[111] E. B. Karbab, M. Debbabi, and A. Derhab, "SwiftR: Cross-platform ransomware fingerprinting using hierarchical neural networks on hybrid features," *Expert Syst. Appl.*, vol. 225, Sep. 2023, Art. no. 120017, doi: 10.1016/j.eswa.2023.120017.

[112] E. Eilam and E. Chikofsky, *Reversing: Secrets of Reverse Engineering*. Hoboken, NJ, USA: Wiley, 2005.

[113] L. Binosi, M. Polino, M. Carminati, and S. Zanero, "BINO: Automatic recognition of inline binary functions from template classes," *Comput. Secur.*, vol. 132, Sep. 2023, Art. no. 103312, doi: 10.1016/j.cose.2023.103312.

[114] Z. Zhuo, G. Tao, G. Shen, S. An, Q. Xu, Y. Liu, Y. Ye, Y. Wu, and X. Zhang, "PELICAN: Exploiting backdoors of naturally trained deep learning models in binary code analysis," in *Proc. 32nd USENIX Secur. Symp. USENIX Secur.*, 2023, pp. 2365–2382.

[115] J. Kinder and H. Veith, "Jakstab: A static analysis platform for binaries—Tool paper," in *Proc. Int. Conf. Comput. Aided Verification*, 2008, pp. 423–427, doi: 10.1007/978-3-540-70545-1_40.

[116] J. Chen, G. Cheng, and H. Mei, "F-ACCUMUL: A protocol fingerprint and accumulative payload length sample-based tor-snowflake traffic-identifying framework," *Appl. Sci.*, vol. 13, no. 1, p. 622, Jan. 2023, doi: 10.3390/app13010622.

[117] I. Agadakos, D. Jin, D. Williams-King, V. P. Kemerlis, and G. Portokalidis, "Nibbler: Debloating binary shared libraries," in *Proc. 35th Annu. Comput. Secur. Appl. Conf.*, Dec. 2019, pp. 70–83, doi: 10.1145/3359789.3359823.

[118] K. Zhang and K. Fenner, "EnviRule: An end-to-end system for automatic extraction of reaction patterns from environmental contaminant biotransformation pathways," *Bioinformatics*, vol. 39, no. 7, p. 407, Jul. 2023, doi: 10.1093/bioinformatics/btad407.

[119] X. Nie, X. Li, Y. Chai, C. Cui, X. Xi, and Y. Yin, "Robust image fingerprinting based on feature point relationship mining," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 6, pp. 1509–1523, Jun. 2018, doi: 10.1109/TIFS.2018.2790953.

[120] J. Löfvenberg, "Binary fingerprinting codes," *Designs, Codes Cryptography*, vol. 36, no. 1, pp. 69–81, Jul. 2005, doi: 10.1007/s10623-003-1163-5.

[121] Y. Zhou, J. Chen, Y. Shi, B. Chen, and Z. M. J. Jiang, "An empirical comparison on the results of different clone detection setups for C-based projects," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng., Softw. Eng. Pract. (ICSE-SEIP)*, May 2023, pp. 74–86, doi: 10.1109/ICSE-SEIP58684.2023.00012.

[122] S. Alrabaee, N. Saleem, S. Preda, L. Wang, and M. Debbabi, "OBA2: An onion approach to binary code authorship attribution," in *Proc. Digit. Forensic Res. Conf. DFRWS EU*, vol. 11, 2014, pp. S94–S103, doi: 10.1016/j.diin.2014.03.012.

[123] P. Shirani, L. Wang, and M. Debbabi, "BinShape: Scalable and robust binary library function identification using function shape," in *Proc. Int. Conf. Detection Intrusions Malware*, 2017, pp. 301–324.

[124] C. Peng, H. Sun, M. Yang, and Y.-L. Wang, "A survey on security communication and control for smart grids under malicious cyber attacks," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 49, no. 8, pp. 1554–1569, Aug. 2019, doi: 10.1109/TSMC.2018.2884952.

[125] S. Kim, T. Zimmermann, K. Pan, and E. J. J. Whitehead, "Automatic identification of bug-introducing changes," in *Proc. 21st IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Sep. 2006, pp. 81–90.

[126] Y. Du, O. Alrawi, K. Snow, M. Antonakakis, and F. Monrose, "Improving security tasks using compiler provenance information recovered at the binary-level," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2023, pp. 2695–2709.

[127] L. Jiang, J. An, H. Huang, Q. Tang, S. Nie, S. Wu, and Y. Zhang, "BinaryAI: Binary software composition analysis via intelligent binary source code matching," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, Apr. 2024, pp. 1–13.

[128] S. Malathi and C. Meena, "An efficient method for partial fingerprint recognition based on local binary pattern," in *Proc. Int. Conf. Commun. CONTROL Comput. Technol.*, Oct. 2010, pp. 569–572, doi: 10.1109/ICCCCT.2010.5670775.

[129] A. Martin, S. Raponi, T. Combe, and R. Di Pietro, "Docker ecosystem—Vulnerability analysis," *Comput. Commun.*, vol. 122, pp. 30–43, Jun. 2018, doi: 10.1016/j.comcom.2018.03.011.

[130] D. Tychalas, H. Benkraouda, and M. Maniatakos, "ICSFuzz: Manipulating I/Os and repurposing binary code to enable instrumented fuzzing in ICS control applications," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2847–2862.

[131] M. Salfer, H. Schweppe, and C. Eckert, "Attack surface and vulnerability assessment of automotive electronic control units," in *Proc. 12th Int. Conf. Secur. Cryptography*, 2015, pp. 317–326, doi: 10.5220/0005550003170326.

[132] Z. Wu, S. Gianvecchio, M. Xie, and H. Wang, "Mimimorphism: A new approach to binary code obfuscation," in *Proc. 17th ACM Conf. Comput. Commun. Secur.*, Oct. 2010, pp. 536–546, doi: 10.1145/1866307.1866368.

[133] S. Chen, Z. Lin, and Y. Zhang, "SelectiveTaint: Efficient data flow tracking with static binary rewriting," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 1665–1682.

[134] R. Amankwah, P. Kwaku, B. Korkor, K. Mensah, B. Brew, and S. Yeboah, "A theoretical framework for software vulnerability detection based on cascaded refinement network," *Int. J. Comput. Appl.*, vol. 182, no. 25, pp. 12–15, Nov. 2018, doi: 10.5120/ijca2018918078.

[135] J. Feist, L. Mounier, S. Bardin, R. David, and M.-L. Potet, "Finding the needle in the heap: Combining static analysis and dynamic symbolic execution to trigger use-after-free," in *Proc. 6th Workshop Softw. Secur., Protection, Reverse Eng.*, Dec. 2016, pp. 1–12, doi: 10.1145/3015135.3015137.

[136] Y. Zheng, A. Davanian, H. Yin, C. Song, H. Zhu, and L. Sun, "FIRM-AFL: High-throughput greybox fuzzing of IoT firmware via augmented process emulation," in *Proc. 28th USENIX Secur. Symp.*, 2019, pp. 1099–1114.

[137] P. Shirani, L. Collard, B. L. Agba, B. Lebel, M. Debbabi, L. Wang, and A. Hanna, "BINARM: Scalable and efficient detection of vulnerabilities in firmware images of intelligent electronic devices," in *Proc. Int. Conf. Detection Intrusions Malware, Vulnerability Assessment*, 2018, pp. 114–138, doi: 10.1007/978-3-319-93411-2_6.

[138] R. Duan, A. Bijlani, Y. Ji, O. Alrawi, Y. Xiong, M. Ike, B. Saltaformaggio, and W. Lee, "Automating patching of vulnerable open-source software versions in application binaries," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, Feb. 2019, pp. 1–12, doi: 10.14722/ndss.2019.23126.

[139] S. Kim and H. Lee, "Software systems at risk: An empirical study of cloned vulnerabilities in practice," *Comput. Secur.*, vol. 77, pp. 720–736, Aug. 2018, doi: 10.1016/j.cose.2018.02.007.

[140] J. Caballero, G. Grieco, M. Marron, and A. Nappa, "Undangle: Early detection of dangling pointers in use-after-free and double-free vulnerabilities," in *Proc. Int. Symp. Softw. Test. Anal.*, Jul. 2012, pp. 133–143, doi: 10.1145/2338965.2336769.

[141] J. Yang, C. Fu, X.-Y. Liu, H. Yin, and P. Zhou, "Codee: A tensor embedding scheme for binary code search," *IEEE Trans. Softw. Eng.*, vol. 48, no. 7, pp. 2224–2244, Jul. 2022, doi: 10.1109/TSE.2021.3056139.

[142] D. Tian, G. Hernandez, J. I. Choi, V. A. Frost, C. Raules, P. Traynor, H. Vijayakumar, L. R. Harrison, A. Rahmati, M. Grace, and K. Butler, "Attention spanned: Comprehensive vulnerability analysis of AT commands within the Android ecosystem," in *Proc. 27th USENIX Secur. Symp.*, 2018, pp. 273–290.

[143] K. A. Roundy and B. P. Miller, "Binary-code obfuscations in prevalent packer tools," *ACM Comput. Surv.*, vol. 46, no. 1, pp. 1–32, Oct. 2013, doi: 10.1145/2522968.2522972.

[144] R. Wartell, V. Mohan, K. W. Hamlen, and Z. Lin, "Securing untrusted code via compiler-agnostic binary rewriting," in *Proc. 28th Annu. Comput. Secur. Appl. Conf.*, Dec. 2012, pp. 299–308, doi: 10.1145/2420950.2420995.

[145] Z. Cui, L. Du, P. Wang, X. Cai, and W. Zhang, "Malicious code detection based on CNNs and multi-objective algorithm," *J. Parallel Distrib. Comput.*, vol. 129, pp. 50–58, Jul. 2019, doi: 10.1016/j.jpdc.2019.03.010.

[146] Y. Chen, M. Khandaker, and Z. Wang, "Pinpointing vulnerabilities," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, Apr. 2017, pp. 334–345, doi: 10.1145/3052973.3053033.

[147] J. Krupp and C. Rossow, "TeEther: Gnawing at Ethereum to automatically exploit smart contracts," in *Proc. 27th USENIX Secur. Symp.*, 2018, pp. 1317–1333.

[148] W. Chang, B. Streiff, and C. Lin, "Efficient and extensible security enforcement using dynamic data flow analysis," in *Proc. 15th ACM Conf. Comput. Commun. Secur.*, Oct. 2008, pp. 39–50, doi: 10.1145/1455770.1455778.

[149] D. Lehmann, J. Kinder, and M. Pradel, "Everything old is new again: Binary security of WebAssembly," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 217–234.

[150] L. Massarelli, G. A. D. Luna, F. Petroni, R. Baldoni, and L. Querzoni, "SAFE: Self-attentive function embeddings for binary similarity," in *Proc. Int. Conf. Detection Intrusions Malware, Vulnerability Assessment*, 2019, pp. 309–329, doi: 10.1007/978-3-030-22038-9_15.

[151] B. Livshits and T. Zimmermann, "DynaMine: Finding common error patterns by mining software revision histories," in *Proc. 10th Eur. Softw. Eng. Conf. 13th ACM SIGSOFT Symp. Found. Softw. Eng.*, 2005, pp. 296–305.

[152] D. Engler, D. Y. Chen, S. Hallem, A. Chou, and B. Chelf, "Bugs as deviant behavior: A general approach to inferring errors in systems code," *ACM SIGOPS Operating Syst. Rev.*, vol. 35, no. 5, pp. 57–72, Dec. 2001, doi: 10.1145/502059.502041.

[153] Z. Yu, X. H. Su, T. T. Wang, and P. J. Ma, "Automatically extracting implicit programming rules and detecting violations from C programs," *Tien Tzu Hsueh Pao/Acta Electron. Sin.*, vol. 41, no. 2, pp. 248–254, 2013, doi: 10.3969/j.issn.0372-2112.2013.02.007.

[154] A. Wasylkowski, A. Zeller, and C. Lindig, "Detecting object usage anomalies," in *Proc. 6th joint meeting Eur. Softw. Eng. Conf. ACM SIGSOFT Symp. Found. Softw. Eng.*, Sep. 2007, pp. 35–44, doi: 10.1145/1287624.1287632.

[155] M. Bishop, "About penetration testing," *IEEE Secur. Privacy Mag.*, vol. 5, no. 6, pp. 84–87, Nov. 2007, doi: 10.1109/MSP.2007.159.

[156] M. Acharya, T. Xie, J. Pei, and J. Xu, "Mining API patterns as partial orders from source code: From usage scenarios to specifications," in *Proc. 6th Joint Meeting Eur. Softw. Eng. Conf. ACM SIGSOFT Symp. Found. Softw. Eng.*, Sep. 2007, pp. 25–34, doi: 10.1145/1287624.1287630.

[157] A. Alzaqebah, I. Aljarah, and O. Al-Kadi, "A hierarchical intrusion detection system based on extreme learning machine and nature-inspired optimization," *Comput. Secur.*, vol. 124, Jan. 2023, Art. no. 102957, doi: 10.1016/j.cose.2022.102957.

[158] A. Pan, C. J. Shern, A. Zou, N. Li, S. Basart, T. Woodside, J. Ng, H. Zhang, S. Emmons, and D. Hendrycks, "Do the rewards justify the means? Measuring trade-offs between rewards and ethical behavior in the MACHIAVELLI benchmark," in *Proc. Mach. Learn. Res.*, 2023, pp. 26837–26867.

[159] V. C. Kanakamedala, S. H. Singh, and R. Talasani, "Sentiment analysis of online customer reviews for handicraft product using machine learning: A case of flipkart," in *Proc. Int. Conf. Advancement Technol. (ICONAT)*, Jan. 2023, pp. 1–6, doi: 10.1109/ICONAT57137.2023.10080169.

[160] J. Gao, X. Yang, Y. Fu, Y. Jiang, H. Shi, and J. Sun, "VulSeekerpro: Enhanced semantic learning based binary vulnerability seeker with emulation," in *Proc. 26th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Oct. 2018, pp. 803–808, doi: 10.1145/3236024.3275524.

[161] H. Yakura, S. Shinozaki, R. Nishimura, Y. Oyama, and J. Sakuma, "Malware analysis of imaged binary samples by convolutional neural network with attention mechanism," in *Proc. 8th ACM Conf. Data Appl. Secur. Privacy*, Mar. 2018, pp. 127–134, doi: 10.1145/3176258.3176335.

[162] K. Cheng, Q. Li, L. Wang, Q. Chen, Y. Zheng, L. Sun, and Z. Liang, "DTaint: Detecting the taint-style vulnerability in embedded device firmware," in *Proc. 48th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, Jun. 2018, pp. 430–441, doi: 10.1109/DSN.2018.00052.

[163] H. Xue, G. Venkataramani, and T. Lan, "Clone-slicer: Detecting domain specific binary code clones through program slicing," in *Proc. Workshop Forming Ecosystem Around Softw. Transformation*, Jan. 2018, pp. 27–33, doi: 10.1145/3273045.3273047.

[164] J. Gao, X. Yang, Y. Fu, Y. Jiang, and J. Sun, "VulSeeker: A semantic learning based vulnerability seeker for cross-platform binary," in *Proc. 33rd IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Sep. 2018, pp. 896–899, doi: 10.1145/3238147.3240480.

[165] J. Salwan, S. Bardin, and M.-L. Potet, "Symbolic deobfuscation: From virtualized code back to the original," in *Proc. Int. Conf. Detection Intrusions Malware, Vulnerability Assessment*, 2018, pp. 372–392, doi: 10.1007/978-3-319-93411-2_17.

[166] A. Nahiyan, F. Farahmandi, P. Mishra, D. Forte, and M. Tehranipoor, "Security-aware FSM design flow for identifying and mitigating vulnerabilities to fault attacks," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 38, no. 6, pp. 1003–1016, Jun. 2019, doi: 10.1109/TCAD.2018.2834396.

[167] Z. Li, D. Zou, S. Xu, Z. Chen, Y. Zhu, and H. Jin, "VulDeeLocator: A deep learning-based fine-grained vulnerability detector," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2821–2837, Jul. 2022, doi: 10.1109/TDSC.2021.3076142.

[168] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?" *IEEE Trans. Softw. Eng.*, vol. 48, no. 9, pp. 3280–3296, Sep. 2022, doi: 10.1109/TSE.2021.3087402.

[169] Z. Zheng, B. Zhang, Y. Liu, J. Ren, X. Zhao, and Q. Wang, "An approach for predicting multiple-type overflow vulnerabilities based on combination features and a time series neural network algorithm," *Comput. Secur.*, vol. 114, Mar. 2022, Art. no. 102572, doi: 10.1016/j.cose.2021.102572.

[170] O. Oleksenko, B. Trach, M. Silberstein, and C. Fetzer, "SpecFuzz: Bringing spectre-type vulnerabilities to the surface," in *Proc. 29th USENIX Secur. Symp.*, 2019, pp. 1481–1498.

[171] D. C. D'Elia, E. Coppa, S. Nicchi, F. Palmaro, and L. Cavallaro, "SoK: Using dynamic binary instrumentation for security (and how you may get caught red handed)," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, Jul. 2019, pp. 15–27, doi: 10.1145/3321705.3329819.

[172] H. Zhang and Z. Qian, "Precise and accurate patch presence test for binaries," in *Proc. 27th USENIX Secur. Symp.*, 2018, pp. 887–902.

[173] M. Li, E. Yumer, and D. Ramanan, "Budgeted training: Rethinking deep neural network training under resource constraints," in *Proc. 8th Int. Conf. Learn. Represent.*, 2020, pp. 1–16.

[174] H. Jang, K. Yang, G. Lee, Y. Na, J. D. Seideman, S. Luo, H. Lee, and S. Dietrich, "QuickBCC: Quick and scalable binary vulnerable code clone detection," in *Proc. IFIP Int. Conf. ICT Syst. Secur. Privacy Protection*, 2021, pp. 66–82, doi: 10.1007/978-3-030-78120-0_5.

[175] K. Tang, F. Liu, Z. Shan, and C. Zhang, "Anti-obfuscation binary code clone detection based on software gene," in *Proc. Int. Conf. Pioneering Comput. Scientists, Eng. Educators*, 2021, pp. 193–208, doi: 10.1007/978-981-16-5940-9_15.

[176] M. Lei, H. Li, J. Li, N. Aundhkar, and D.-K. Kim, "Deep learning application on code clone detection: A review of current knowledge," *J. Syst. Softw.*, vol. 184, Feb. 2022, Art. no. 111141, doi: 10.1016/j.jss.2021.111141.

[177] G. Kaur and E. S. Sharma, "The survey of the code clone detection techniques and process with types ( I, II, III and IV)," *Int. J. Futur. Revolut. Comput. Sci. Commun. Eng.*, vol. 4, no. 2, pp. 392–399, 2018.

[178] M. Raginsky and S. Lazebnik, "Locality-sensitive binary codes from shift-invariant kernels," in *Proc. Conf. Adv. Neural Inf. Process. Syst.*, vol. 22, 2009, pp. 1509–1517.

[179] S. Alrabaee, "A stratified approach to function fingerprinting in program binaries using diverse features," *Expert Syst. Appl.*, vol. 193, May 2022, Art. no. 116384, doi: 10.1016/j.eswa.2021.116384.

[180] M. Mondal, B. Roy, C. K. Roy, and K. A. Schneider, "An empirical study on bug propagation through code cloning," *J. Syst. Softw.*, vol. 158, Dec. 2019, Art. no. 110407, doi: 10.1016/j.jss.2019.110407.

[181] A. Gupta and B. Suri, "A survey on code clone, its behavior and applications," in *Networking Communication and Data Knowledge Engineering: Volume 2* (Lecture Notes on Data Engineering and Communications Technologies). Cham, Switzerland: Springer, 2018, pp. 27–39, doi: 10.1007/978-981-10-4600-1_3.

[182] N. Saini, S. Singh, and Suman, "Code clones: Detection and management," *Proc. Comput. Sci.*, vol. 132, pp. 718–727, Jan. 2018.

[183] Y. Zhang, A. X. Xu, and Y. Jiang, "Scalable and accurate binary code search method based on simhash and partial trace," in *Proc. IEEE 19th Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, Dec. 2020, pp. 818–826, doi: 10.1109/TrustCom50675.2020.00111.

[184] D. Keltner, D. Sauter, J. Tracy, and A. Cowen, "Emotional expression: Advances in basic emotion theory," *J. Nonverbal Behav.*, vol. 43, no. 2, pp. 133–160, Jun. 2019, doi: 10.1007/s10919-019-00293-3.

[185] Y. Namiki, T. Ishida, and Y. Akiyama, "Acceleration of sequence clustering using longest common subsequence filtering," *BMC Bioinf.*, vol. 14, no. S8, pp. 1–8, May 2013, doi: 10.1186/1471-2105-14-s8-s7.

[186] L. Buch and A. Andrzejak, "Learning-based recursive aggregation of abstract syntax trees for code clone detection," in *Proc. IEEE 26th Int. Conf. Softw. Anal., Evol. Reengineering (SANER)*, Feb. 2019, pp. 95–104, doi: 10.1109/SANER.2019.8668039.

[187] M. R. Farhadi, B. C. M. Fung, Y. B. Fung, P. Charland, S. Preda, and M. Debbabi, "Scalable code clone search for malware analysis," *Digit. Invest.*, vol. 15, pp. 46–60, Dec. 2015, doi: 10.1016/j.diin.2015.06.001.

[188] H. Huang, "Binary code reuse detection for reverse," Master's thesis, Concordia Univ., Montréal, QC, Canada, 2016.

[189] M. R. Farhadi, "Assembly code clone detection for malware," M.A.Sc. thesis, Concordia Univ., Montreal, QC, Canada, 2013.

[190] M. Nayrolles and A. Hamou-Lhadj, "CLEVER: Combining code metrics with clone detection for just-in-time fault prevention and resolution in large industrial projects," in *Proc. 15th Int. Conf. Mining Softw. Repositories*, May 2018, pp. 153–164, doi: 10.1145/3196398.3196438.

[191] J. Kim and J. M. Youn, "Malware behavior analysis using binary code tracking," in *Proc. 4th Int. Conf. Comput. Appl. Inf. Process. Technol. (CAIPT)*, Aug. 2017, pp. 1–4.

[192] A. M. Sheneamer, "Multiple similarity-based features blending for detecting code clones using consensus-driven classification," *Expert Syst. Appl.*, vol. 183, Nov. 2021, Art. no. 115364, doi: 10.1016/j.eswa.2021.115364.

[193] D. Yu, J. Yang, X. Chen, and J. Chen, "Detecting Java code clones based on bytecode sequence alignment," *IEEE Access*, vol. 7, pp. 22421–22433, 2019, doi: 10.1109/ACCESS.2019.2898411.

[194] R. Xu and D. WunschII, "Survey of clustering algorithms," *IEEE Trans. Neural Netw.*, vol. 16, no. 3, pp. 645–678, May 2005, doi: 10.1109/TNN.2005.845141.

[195] M. Sotoodeh, M. R. Moosavi, and R. Boostani, "A structural based feature extraction for detecting the relation of hidden substructures in coral reef images," *Multimedia Tools Appl.*, vol. 78, no. 24, pp. 34513–34539, Dec. 2019, doi: 10.1007/s11042-019-08050-w.

[196] Y. T. Ling, N. F. M. Sani, M. T. Abdullah, and N. A. W. A. Hamid, "Structural features with nonnegative matrix factorization for metamorphic malware detection," *Comput. Secur.*, vol. 104, May 2021, Art. no. 102216, doi: 10.1016/j.cose.2021.102216.

[197] U. Urooj, B. A. S. Al-Rimy, A. Zainal, F. A. Ghaleb, and M. A. Rassam, "Ransomware detection using the dynamic analysis and machine learning: A survey and research directions," *Appl. Sci.*, vol. 12, no. 1, p. 172, Dec. 2021, doi: 10.3390/app12010172.

[198] M. Wiegmann, B. Stein, and M. Potthast, "Overview of the celebrity profiling task at PAN 2020," in *Proc. CEUR Workshop*, 2020, pp. 9–12.

[199] K. Lagutina, N. Lagutina, E. Boychuk, I. Vorontsova, E. Shliakhtina, O. Belyaeva, I. Paramonov, and P. G. Demidov, "A survey on stylometric text features," in *Proc. 25th Conf. Open Innov. Assoc. (FRUCT)*, Nov. 2019, pp. 184–195, doi: 10.23919/FRUCT48121.2019.8981504.

[200] J. Ngadiuba, V. Loncar, M. Pierini, S. Summers, G. Di Guglielmo, J. Duarte, P. Harris, D. Rankin, S. Jindariani, M. Liu, K. Pedro, N. Tran, E. Kreinar, S. Sagear, Z. Wu, and D. Hoang, "Compressing deep neural networks on FPGAs to binary and ternary precision with hls4ml," *Mach. Learn., Sci. Technol.*, vol. 2, no. 1, Dec. 2020, Art. no. 015001, doi: 10.1088/2632-2153/aba042.

[201] L. Demetrio, B. Biggio, G. Lagorio, F. Roli, and A. Armando, "Explaining vulnerabilities of deep learning to adversarial malware binaries," in *Proc. CEUR Workshop*, vol. 2315, 2019, pp. 1–13.

[202] G. Kälin, Z. Liu, and R. A. Porto, "Conservative tidal effects in compact binary systems to next-to-leading post-Minkowskian order," *Phys. Rev. D, Part. Fields*, vol. 102, no. 12, Dec. 2020, Art. no. 124025, doi: 10.1103/physrevd.102.124025.

[203] J. Too, A. R. Abdullah, and N. M. Saad, "A new quadratic binary Harris hawk optimization for feature selection," *Electronics*, vol. 8, no. 10, pp. 1–27, Oct. 2019, doi: 10.3390/electronics8101130.

[204] N. Potha and E. Stamatatos, "Intrinsic author verification using topic modeling," in *Proc. 10th Hellenic Conf. Artif. Intell.*, Jul. 2018, pp. 1–7, doi: 10.1145/3200947.3201013.

[205] M. Benthin, "Attributing malware binaries to threat actors based on authorship style," Ruhr-Universität Bochum, 2022, doi: 10.13140/RG.2.2.10524.21128.

[206] D. S. Trigueros, L. Meng, and M. Hartnett, "Face recognition: From traditional to deep learning methods," 2018, *arXiv:1811.00116*.

[207] S. Zaharia, T. Rebedea, and S. Trausan-Matu, "Detection of software security weaknesses using cross-language source code representation (CLaSCoRe)," *Appl. Sci.*, vol. 13, no. 13, p. 7871, Jul. 2023, doi: 10.3390/app13137871.

[208] W. Godoy, P. Valero-Lara, K. Teranishi, P. Balaprakash, and J. Vetter, "Evaluation of OpenAI codex for HPC parallel programming models kernel generation," in *Proc. 52nd Int. Conf. Parallel Process. Workshops*, Aug. 2023, pp. 136–144, doi: 10.1145/3605731.3605886.

[209] B. Lecoeur, H. Mohsin, and A. F. Donaldson, "Program reconditioning: Avoiding undefined behaviour when finding and reducing compiler bugs," *Proc. ACM Program. Lang.*, vol. 7, pp. 1801–1825, Jun. 2023, doi: 10.1145/3591294.

[210] W. Daelemans, M. Kestemont, E. Manjavacas, M. Potthast, F. Rangel, P. Rosso, G. Specht, E. Stamatatos, B. Stein, M. Tschuggnall, M. Wiegmann, and E. Zangerle, "Overview of PAN 2019: Bots and gender profiling, celebrity profiling, cross-domain authorship attribution and style change detection," in *Proc. Int. Conf. Cross-Lang. Eval. Forum Eur. Lang.*, 2019, pp. 402–416, doi: 10.1007/978-3-030-28577-7_30.

[211] Y. Hu, Y. Zhang, J. Li, H. Wang, B. Li, and D. Gu, "BinMatch: A semantics-based hybrid approach on binary code clone analysis," in *Proc. IEEE Int. Conf. Softw. Maintenance Evol. (ICSME)*, Sep. 2018, pp. 104–114, doi: 10.1109/ICSME.2018.00019.

[212] F. Jafariakinabad and K. A. Hua, "A self-supervised representation learning of sentence structure for authorship attribution," *ACM Trans. Knowl. Discovery Data*, vol. 16, no. 4, pp. 1–16, Aug. 2022, doi: 10.1145/3491203.

[213] S. Myronenko and Y. Myronenko, "Detecting plagiarism in student assignments using source code analysis," *WSEAS Trans. Comput. Res.*, vol. 12, pp. 367–376, Sep. 2024.

[214] W. Zheng and M. Jin, "A review on authorship attribution in text mining," *WIREs Comput. Statist.*, vol. 15, no. 2, p. 1584, Mar. 2023, doi: 10.1002/wics.1584.

[215] R. Sayed, H. Azmi, H. Shawkey, A. H. Khalil, and M. Refky, "A systematic literature review on binary neural networks," *IEEE Access*, vol. 11, pp. 27546–27578, 2023, doi: 10.1109/ACCESS.2023.3258360.

[216] F. Alqahtani and M. Dohler, "Survey of authorship identification tasks on Arabic texts," *ACM Trans. Asian Low-Resource Lang. Inf. Process.*, vol. 22, no. 4, pp. 1–24, Apr. 2023, doi: 10.1145/3564156.

[217] S. Al Atiiq, C. Gehrmann, and K. Dahlén, "Vulnerability detection in popular programming languages with language models," 2024, *arXiv:2412.15905*.

[218] N. Galloro, M. Polino, M. Carminati, A. Continella, and S. Zanero, "A systematical and longitudinal study of evasive behaviors in windows malware," *Comput. Secur.*, vol. 113, Feb. 2022, Art. no. 102550, doi: 10.1016/j.cose.2021.102550.

[219] I. N. A. Asmad, A.-F. Mubarak Ali, and N. I. S. I. Jailani, "Enhancement of generic code clone detection model for Python application," *Int. J. Softw. Eng. Comput. Syst.*, vol. 8, no. 1, pp. 1–10, Jan. 2022, doi: 10.15282/ijsecs.8.1.2022.1.0092.

[220] Y. Gao, Z. Lu, and Y. Luo, "Survey on malware anti-analysis," in *Proc. 5th Int. Conf. Intell. Control Inf. Process.*, Aug. 2014, pp. 270–275, doi: 10.1109/ICICIP.2014.7010353.

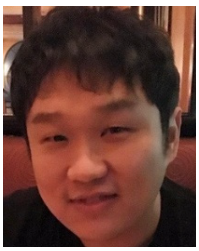
[221] H. Javed, S. El-Sappagh, and T. Abuhmed, "Robustness in deep learning models for medical diagnostics: Security and adversarial challenges towards robust AI applications," *Artif. Intell. Rev.*, vol. 58, no. 1, pp. 1–107, Nov. 2024, doi: 10.1007/s10462-024-11005-9.

[222] M. Kim, H. Cho, and J. H. Yi, "Large-scale analysis on anti-analysis techniques in real-world malware," *IEEE Access*, vol. 10, pp. 75802–75815, 2022, doi: 10.1109/ACCESS.2022.3190978.
[223] Y. Kawakoya, E. Shioji, M. Iwamura, and J. Miyoshi, "API chaser: Taint-assisted sandbox for evasive malware analysis," *J. Inf. Process.*, vol. 27. Cham, Switzerland: Springer, pp. 297–314, Jan. 2019, doi: 10.2197/ipsjjip.27.297.
[224] F. Kreuk, A. Barak, S. Aviv-Reuven, M. Baruch, B. Pinkas, and J. Keshet, "Deceiving end-to-end deep learning malware detectors using adversarial examples," 2018, *arXiv:1802.04528*.
[225] J. Cabrera-Arteaga, M. Monperrus, T. Toady, and B. Baudry, "WebAssembly diversification for malware evasion," *Comput. Secur.*, vol. 131, Aug. 2023, Art. no. 103296, doi: 10.1016/j.cose.2023.103296.
[226] L. Vinet and A. Zhedanov, "A 'missing' family of classical orthogonal polynomials," *J. Phys. A, Math. Theor.*, vol. 44, no. 8, Feb. 2011, Art. no. 085201, doi: 10.1088/1751-8113/44/8/085201.
[227] A. Singh and K. Chatterjee, "Cloud security issues and challenges: A survey," *J. Netw. Comput. Appl.*, vol. 79, pp. 88–115, Feb. 2017, doi: 10.1016/j.jnca.2016.11.027.
[228] M. Brengel, M. Backes, and C. Rossow, "Detecting hardware-assisted virtualization," in *Proc. 13th Int. Conf. Detection Intrusions Malware, Vulnerability Assessment*, 2016, pp. 207–227.
[229] L. S. Leenu Singh and S. I. Hassan, "Virtualization evolution for transparent malware analysis," *Int. J. Sci. Res.*, vol. 2, no. 6, pp. 101–104, Jun. 2012, doi: 10.15373/22778179/june2013/33.
[230] M. Polino, A. Continella, S. Mariani, S. D'Alessio, L. Fontana, F. Gritti, and S. Zanero, "Measuring and defeating anti-instrumentation-equipped malware," in *Proc. Int. Conf. Detection Intrusions Malware, Vulnerability Assessment*, 2017, pp. 73–96, doi: 10.1007/978-3-319-60876-1_4.
[231] Z. Luo, T. Rezk, and M. Serrano, "Automated code injection prevention for Web applications," in *Proc. Joint Workshop Theory Secur. Appl.*, 2012, pp. 186–204, doi: 10.1007/978-3-642-27375-9_11.
[232] K. C. S. Gaurav, A. D. Keromytis, and V. Prevelakis, "Countering code-injection attacks with instruction-set randomization," in *Proc. 10th ACM Conf. Comput. Commun. Secur.*, Oct. 2003, pp. 272–280.
[233] M. Oreyomi and H. Jahankhani, "Challenges and opportunities of autonomous cyber defence (ACyD) against cyber attacks," in *Blockchain and Other Emerging Technologies for Digital Business Strategies* (Advanced Sciences and Technologies for Security Applications), 2022, pp. 239–269, doi: 10.1007/978-3-030-98225-6_9.
[234] H. Zhang, B. Li, W. Li, L. Zhu, C. Chang, and S. Yu, "MRCIF: A memory-reverse-based code injection forensics algorithm," *Appl. Sci.*, vol. 13, no. 4, p. 2478, Feb. 2023, doi: 10.3390/app13042478.
[235] Y. Choi, Y. Jeong, D. Jang, B. B. Kang, and H. Lee, "EmuID: Detecting presence of emulation through microarchitectural characteristic on ARM," *Comput. Secur.*, vol. 113, Feb. 2022, Art. no. 102569, doi: 10.1016/j.cose.2021.102569.
[236] G. Morse, M. Alqaradaghi, and T. Kozsik, "Control flow obfuscation with irreducible loops and self-modifying code," *Publicationes Mathematicae Debrecen*, vol. 100, pp. 617–637, Dec. 2022, doi: 10.5486/pmd.2022.suppl.5.

**HASEEB JAVED** (Member, IEEE) received the B.Sc. degree in electrical engineering from the University of Engineering and Technology, Taxila, Pakistan, in 2019, and the M.Sc. degree in electrical engineering from the Muhammad Nawaz Sharif University of Engineering and Technology (MNS UET), Multan, Pakistan, in 2021. He is currently pursuing the Ph.D. degree in computer science and engineering with Sungkyunkwan University (SKKU), Seoul, South Korea. Since 2022, he has been a Ph.D. Candidate with SKKU, focusing on adversarial machine learning, binary code analysis, and applying large language models for software vulnerability detection and reverse engineering. His primary research interests include artificial intelligence, machine learning, natural language processing (NLP), and cybersecurity. His research also involves advanced NLP and text analysis for cybersecurity, with several published papers on NLP, text mining, and scalable AI-based security frameworks.

**FARMAN ALI** received the Bachelor of Science degree in computer science from the University of Peshawar, Pakistan, in 2011, the Master of Science degree in computer science from Gyeongsang National University, South Korea, in 2015, and the Doctor of Philosophy degree in information and communication engineering from Inha University, South Korea, in 2018. He was a Postdoctoral Fellow with the UWB Wireless Communications Research Center, Inha University, from September 2018 to August 2019. He is currently an Associate Professor with the Department of Applied AI, Sungkyunkwan University, South Korea. He has filed over four patents and published more than 100 research papers in peer-reviewed international journals and conferences. His research interests include sentiment analysis, social network analysis, medical informatics, machine learning, artificial intelligence, recommendation systems, data science, and applied fuzzy logic. He received the Outstanding Research Award (Excellence of Journal Publications-2017) and the President's Choice of the Best Researcher Award during his graduate studies at Inha University. In 2022 and 2023, Stanford University named him among the "TOP 2% Scientists in the World" for his career achievements.

**BABAR SHAH** is currently an Associate Professor with the College of Technological Innovation, Zayed University, Abu Dhabi Campus, United Arab Emirates. He is deeply involved in various professional activities, including guest editorships, leading workshops, participating in technical program committees, and reviewing for leading international journals and conferences. His work focuses on improving secure, efficient, and intelligent networked systems through innovations in network scalability, data privacy, and adaptive communication protocols. His research interests include wireless sensor networks (WSN), wireless body area networks (WBAN), the Internet of Things (IoT), churn prediction, security, real-time communication, mobile peer-to-peer (P2P) networks, and mobile learning (M-learning). Additionally, he explores the integration of these technologies to develop comprehensive solutions that address current technological challenges.



**DAEHAN KWAK** (Senior Member, IEEE) received the M.S. degree from Korea Advanced Institute of Science and Technology (KAIST), South Korea, in 2008, and the Ph.D. degree in computer science from Rutgers University, New Brunswick, NJ, USA, in 2017. He was affiliated with the Networked Systems and Security (Disco) Laboratory, Rutgers University; and the Telematics and USN Research Division, Electronics and Telecommunications Research Institute (ETRI). He was a Research Staff Member with the UWB Wireless Research Center, Inha University; the Wireless Internet and Networks Laboratory, KAIST; and the Network Management and Optimization Laboratory, Yonsei University. He is currently an Associate Professor with the Department of Computer Science and Technology, Kean University, NJ, USA. His research interests include applied AI, deep learning, intelligent systems, cyber-physical systems, the Internet of Things (IoT), systems and networking, wireless and sensor systems, mobile and vehicular computing, smart transportation, and smart health. He is a reviewer of several international journals and conferences, and has also served as a guest editor for various journals.